MINOR PROJECT REPORT

PROOFPAY: A BLOCKCHAIN-BASED E-COMMERCE PAYMENT PROTECTION SYSTEM USING CHAINLINK CRE

Conditional ERC-20 escrow, off-chain shipment verification and automated settlement on a local Hardhat network

Submitted by	[Student Name 1] — [USN]
	[Student Name 2] — [USN]
	[Student Name 3] — [USN]
Department	Department of Computer Science and Engineering
Institution	[University / Institute Name]
Academic year	2025 – 2026
Project guide	[Guide / Faculty Name], [Designation]
Technology	Solidity 0.8.28 · Hardhat 2.29.1 · React 19 · Express 4 · Chainlink CRE SDK 1.23.0

Institution logo omitted deliberately: no authentic institutional image asset was available in the repository, and no substitute was generated. All names, USNs and guide details above are placeholders to be completed by the authors.

TABLE OF CONTENTS

Chapter	Page
1. Introduction	04
2. Objectives	06
3. Problem Statement and Domain Linkage	07
3.1 Problem statement	07
3.2 Linkage to FinTech, blockchain and an external data source	07
3.3 Details of the external source	08
4. Literature Review	09
4.1 Review of selected work	09
4.2 Summary of the literature	09
4.3 Research gap and positioning of ProofPay	10
5. Existing System	11
6. Proposed System	13
7. Technologies Used	15
8. Architecture Diagram	16
9. Features	19
10. Implementation	22
10.1 Requirement analysis and design	22
10.2 Smart contract development	22
10.3 Compilation and testing	24
10.4 Local blockchain deployment	24
10.5 Frontend development	24
10.6 Backend development	24
10.7 Chainlink CRE workflow development	25
10.8 Local execution bridge	26
10.9 Automatic courier demonstration	27
10.10 Smart contract design summary	27
10.11 Key code snippets	29
10.12 Deployment details	30

Chapter (continued) **Page**

11. Results 32
11.1 Smart contract test results 32
11.2 Backend test results 32
11.3 Frontend results 33
11.4 Actual blockchain settlement results 33
11.5 Test cases 36
11.6 Gas analysis 37
11.7 Screenshots and visual results 38
12. Security Analysis and Limitations 46
13. Conclusion 48
14. Future Scope 49
15. References 50
Appendix A — Deployment and Transaction Ledger 51
Appendix B — List of Figures and Tables 52

Page numbers refer to the physical pages of this PDF, which are numbered sequentially from the title page in the footer.

1. INTRODUCTION

Electronic commerce has grown into the default channel for buying goods that the buyer cannot inspect before paying. That growth has produced a specific and stubborn asymmetry: the party who must act first is the party with the least information. A buyer is asked to transfer value for a product that has not yet been shipped, from a seller whose fulfilment record cannot be verified, and to a logistics chain that the buyer does not control. For low-value goods the asymmetry is absorbed by volume — marketplaces refund first and investigate later. For high-value goods, where a single transaction may be worth a month of income, it becomes the thing that stops the purchase.

The risks are well understood. The seller may never ship. The parcel may be lost, damaged in transit, or delivered to the wrong address. The tracking record may show "delivered" when nothing arrived, or may stall indefinitely in a transit state that nobody resolves. Payment disputes of this kind are rarely questions of law; they are questions of evidence, and the evidence lives in three incompatible places: the payment provider's ledger, the marketplace's order database, and the courier's tracking system. Reconciling them takes days and depends on a human adjudicator who is trusted to read all three honestly.

Traditional escrow solves the custody half of the problem by inserting a neutral intermediary. The buyer's money is held by a third party and released when a condition is met. This is mature practice for property and legal settlements, but it is heavy machinery: onboarding is manual, fees scale with value, and the condition — "was it delivered?" — is still evaluated by a person reading a tracking page. The intermediary is also a single point of both failure and trust: the parties must believe that the funds exist, that the rule will be applied as written, and that the record of what happened is accurate. None of those beliefs is independently checkable.

Blockchain changes the custody half again, more decisively. A smart contract can hold an ERC-20 balance that no operator can spend, apply a rule that is fixed in bytecode, and publish every state change as an event that anyone can read and no one can alter retroactively. For a payment-protection system this is a genuine structural improvement: the buyer's funds are provably locked, the release condition is public, and the resulting settlement is auditable years later from a block explorer alone. What the blockchain cannot do is observe the world. A contract has no way to know that a parcel arrived; it can only be told, and whoever tells it holds the power that the escrow intermediary used to hold.

This is the oracle problem, and it is the central technical difficulty of any shipment-linked payment system. It is also why ProofPay is built around Chainlink CRE rather than around a contract with an administrative settlement function. Chainlink's Compute Runtime Environment lets a workflow fetch data over HTTP, read chain state over EVM, apply arbitrary logic, reach consensus across independent nodes, and produce a report that a receiver contract can verify and act on. A shipment-tracking API is exactly the kind of off-chain source that a contract cannot reach on its own, and a CRE workflow is exactly the kind of component that can reach it, combine it with on-chain state, and deliver a decision with provenance.

ProofPay applies this combination to e-commerce payment protection. A buyer funds a smart-contract escrow with a dedicated ERC-20 token, ProofPay USD. The escrow holds the funds and exposes exactly two settlement functions: one that pays the seller, one that repays the buyer, both restricted to a single oracle address. A separate Express service stores shipment records, which are written by the seller and advanced by a simulated courier. A CRE workflow triggered on a schedule fetches those records over HTTP, reads the corresponding escrow over EVM, decides whether the shipment's status justifies settlement, and encodes a report containing the order ID, the outcome, a hash of the tracking ID and the observation time. A receiver contract validates the report's authorisation, its uniqueness and the escrow's funding state, and then calls the escrow.

Two design decisions distinguish the implementation described in this report from a generic escrow demo. The first is that the settlement decision is made in exactly one place. The workflow owns the mapping from shipment status to outcome; the bridge that carries the report to the chain owns no rules at all and cannot change the outcome, only decline to relay it. The second is honesty about the execution environment. The installed CRE SDK does not expose a chain selector for a local Hardhat node, and no deployed DON was available, so the workflow runs through the official CRE CLI simulator with the on-chain write disabled, and a local bridge submits the report it produced. That is real

CRE execution — real HTTP fetch, real EVM read, real report encoding — combined with a local relay in place of DON consensus. The distinction matters and is maintained throughout this report rather than glossed over.

The result is a system that behaves like a financial protection product rather than a cryptocurrency demonstration. Value is denominated in a token that represents a fiat claim; gas is paid separately in test ETH and never confused with the amount under protection; the release condition is a courier's record rather than an administrator's opinion; and both outcomes of a purchase — payment and refund — travel through the same automated pipeline. The remainder of this report sets out the objectives, situates the work in the research literature, describes the existing approaches it improves on, documents the architecture and implementation in detail, and reports the measured results of thirty-one contract tests, seventeen backend tests, a clean front-end build, and three end-to-end settlements executed on chain with verified receipts.

2. OBJECTIVES

The objectives below were fixed before implementation and are stated so that each can be checked against an artefact in the repository or a transaction on the chain. Every one of them is met by the delivered system; the evidence is cited in Section 11.

1. **Implement a blockchain-based escrow mechanism.** Provide a contract that holds a buyer's payment, records the parties and the amount, and exposes only two terminal transitions, release and refund. *Met by* `ProofPayEscrow` with its `Escrow` struct and four-state enum.
2. **Develop a custom ERC-20 payment token.** Issue a fiat-denominated unit for the demonstration so that escrowed value is separate from gas. *Met by* `ProofPayUSD` (pUSD, 18 decimals, owner-only mint).
3. **Allow buyers to securely fund escrow orders.** Require an explicit allowance and a single funding call, with every validation enforced on chain. *Met by* `approve` followed by `createEscrow`, driven from the checkout page.
4. **Prevent premature access to escrowed funds.** Ensure that neither party, nor the owner, can withdraw before the delivery condition is satisfied. *Met by* the absence of any withdrawal path and by `onlyOracle` on both settlement functions.
5. **Integrate external shipment information.** Maintain shipment status outside the blockchain in a service that the chain cannot modify. *Met by* the Express shipment API with validated transitions and JSON persistence.
6. **Use Chainlink CRE to fetch and process off-chain shipment data.** Perform the fetch inside the workflow, not in the bridge or the front end. *Met by* `HTTPClient.sendRequest` with `consensusIdenticalAggregation`.
7. **Determine delivery and lost-shipment settlement outcomes.** Implement the full status-to-outcome mapping, including the states that must not settle. *Met by* `shouldSettle`, whose six branches are exercised in the CRE logs.
8. **Implement a receiver contract for authorised report execution.** Validate the caller, decode the report, and prevent replay. *Met by* `ProofPayCREReceiver` with `onlyAuthorized`, `processedReports` and `orderSettled`.
9. **Automate seller payments and buyer refunds through the local CRE execution bridge.** Relay CRE's own bytes to the chain and verify the receipt. *Met by* `local-runner.ts` and three verified settlements at blocks 24, 29 and 30.
10. **Provide separate buyer, seller and admin dashboards.** Give each role a view built from on-chain state. *Met by* the React application with `useEscrows`, the seller view and the admin panels.
11. **Prevent duplicate settlements and unauthorised transactions.** Reject a repeated report, a second settlement of one order, and any caller that is not authorised. *Met by* two receiver mappings, the escrow's funding check, the bridge's state file and its order allowlist.
12. **Validate the system through automated tests and actual blockchain transactions.** Run the suites and then settle real orders. *Met by* 31 contract tests, 17 backend tests, a clean TypeScript build, and receipts with status 1.

3. PROBLEM STATEMENT AND DOMAIN LINKAGE

3.1 Problem statement

Buyers and sellers in high-value online commerce must transact without a shared source of truth about delivery, so the buyer risks paying for goods that never arrive while the seller risks shipping goods that are never paid for, and both depend on a platform or escrow agent whose custody, rules and records they cannot independently verify. Existing protections replace that uncertainty with institutional trust and slow human adjudication rather than with verifiable automation. ProofPay addresses the gap by combining a conditional ERC-20 escrow, a separate off-chain shipment record, and a Chainlink CRE workflow that fetches the shipment data, verifies the escrow state, and produces the settlement report that an authorised receiver contract executes on chain.

3.2 Linkage to FinTech, blockchain and an external data source

Table 1 sets out how the problem maps onto the three domains the project draws from. The FinTech column explains why the problem is a financial one and who is exposed; the blockchain column states exactly which operations the project puts on chain; the external-source column explains why shipment data cannot live inside the contract and how it is brought in.

Table 1: Linkage of the Problem Statement

Linked to	Team's specification
FinTech	ProofPay is a digital-payments system with escrow-based transaction protection. Its subject matter is e-commerce payment security: conditional release of funds on delivery, automatic refund on loss, and financial protection for both the buyer (who would otherwise prepay) and the merchant (who would otherwise ship unsecured). The financial stakeholders are the buyer, who supplies the principal; the seller, who supplies the goods and bears the settlement delay; and the oracle operator, who supplies evidence but holds no funds. The transaction process is: the buyer's pUSD is debited into escrow, held for the duration of fulfilment, then credited either to the seller's pUSD balance or back to the buyer's, with gas paid separately in test ETH so that the protected amount is never consumed by fees. Because settlement is conditional rather than immediate, the system also defines what happens when the condition is never met — an escrow can remain `Funded` indefinitely, which is a deliberate safety property rather than an oversight, and the admin path exists to resolve such cases.
Blockchain	Blockchain is used because the guarantees the system sells are custody, finality and auditability. Custody: pUSD is held by `ProofPayEscrow`, and no externally owned account can move it — there is no withdrawal, sweep or pause function. Immutability and transparency: every settlement is a transaction whose input, events and resulting balances are permanently readable; the escrow's `EscrowCreated`, `EscrowReleased` and `EscrowRefunded` events, and the receiver's `ReportProcessed` and `SettlementExecuted` events, form an audit trail that no participant can rewrite. Automation: the release condition is code (`onlyOracle` plus a check that the escrow is `Funded`), not a policy applied by staff. Decentralised verification potential: the receiver accepts reports from a forwarder address, which is the hook a deployed CRE DON would use, so the design is not tied to the local relay. Token payments: value moves as ERC-20 transfers of pUSD, which makes amounts exact and comparable. Event logging and role-based authorisation: `Ownable`, `onlyOwner`, `onlyOracle` and `onlyAuthorized` separate configuration from settlement. Concretely, the operations that happen on chain are: token minting by the owner, allowance and escrow funding, escrow state transitions, the two payout transfers, and the recording of report IDs and settled orders. Shipment status, courier identity, product names and timestamps of physical events do *not* happen on chain.
External source (live / off-chain data)	The external source is the ProofPay shipment API, an Express service that stores one record per order with a tracking ID, carrier label, status, timestamps and an event history. It is consumed by the CRE workflow over HTTP; the workflow reads the full collection, filters for orders that are eligible, and then verifies each against the escrow. The reason this data cannot simply be assumed to exist inside a smart contract is that a contract has no network access and no notion of a parcel: it cannot poll a courier, it cannot parse a tracking page, and even if it could, the data would have to be pushed by someone with a key. Keeping the record off chain also keeps it correctable — a mis-scanned parcel is a normal operational event, whereas an on-chain write is permanent. The consequence is that the chain must be told the truth by a component it trusts, which is precisely the role the CRE workflow and the receiver's authorisation checks are designed to fill.

3.3 Details of the external source

Table 2 records the specifics of the integration as implemented, including its limits. The endpoint is a local development address, not a public carrier API, and the report states that plainly: no integration with DHL, FedEx, UPS or any other commercial carrier exists in the codebase. What exists is a service with the shape of a carrier API — a collection endpoint, a single-record endpoint, authenticated mutations and a status vocabulary — which is what a production integration would also present.

Table 2: Details of the External Source

Item	Details
1. Name and URL of source	ProofPay Shipment API (Express 4.21.2), base URL `http://localhost:5001`. Collection endpoint `GET /api/shipments`; single record `GET /api/shipments/:orderId`; registration `POST /api/shipments`; status change `PATCH /api/shipments/:orderId/status`. The CRE staging configuration points at this base URL with path `/api/shipments`. This is the local development endpoint only.
2. Type of data	Off-chain shipment data: order ID (32-byte hex), tracking ID (string), carrier label, status from the set CREATED / SHIPPED / IN_TRANSIT / DELIVERED / LOST / RETURNED, ISO-8601 timestamps for creation and last update, and an append-only event list with a note per transition. No personal data, no addresses and no payment data are stored.
3. How the data reaches the contract	Courier simulator or seller action → `PATCH /api/shipments/:orderId/status` → JSON store → CRE workflow HTTP GET → `shouldSettle` → ABI-encoded report → `PROOFPAY_REPORT_V1` log record → local execution bridge → `ProofPayCREReceiver.processReport(metadata, report)` → `confirmDelivery` or `confirmFailure` on the escrow. No contract ever reads the API directly.
4. Update frequency	Event-driven for seller actions; the demo courier scheduler advances a shipment after 10 s (CREATED→SHIPPED), 10 s (SHIPPED→IN_TRANSIT) and 20 s (IN_TRANSIT→DELIVERED or LOST), with a 3 s poll interval while work is pending. The CRE workflow is declared with the cron expression `0 */5 * * * *`, so a deployed schedule would evaluate shipments every five minutes; in simulation the trigger fires once per invocation. No real-time guarantee is claimed, and the observed end-to-end latency in the demonstration was governed by how often the bridge was invoked, not by the cron expression.
5. Trust and security considerations	Incorrect shipment data (a wrong status, or an order linked to the wrong tracking ID) would produce a wrong settlement, so the receiver independently re-checks that the escrow is still `Funded` before paying. Delayed updates can leave an escrow funded after delivery; the design tolerates this because a late settlement is recoverable while a duplicate one is not. Manipulated status is possible for anyone holding the mutation secret, so the secret is server-side only, mutating routes reject requests without it, and status transitions are validated rather than accepted verbatim. API availability is a liveness dependency: if the API is down the workflow logs an HTTP error and settles nothing, which fails safe. Report replay is blocked on chain by `processedReports`, and unauthorised settlement is blocked by `onlyAuthorized` on the receiver and `onlyOracle` on the escrow. The strongest honest statement is that the trust boundary sits at the API and the relay: a compromised API plus a compromised relay could produce a wrong but well-formed settlement.
6. Where it is used in the contract	The contract never sees the shipment record. It sees only the report produced from it: `abi.decode(report, (bytes32, string, bytes32, string))` yields the order ID, the outcome string, the `keccak256` hash of the tracking ID and the observation timestamp. The order ID is used to look up the escrow (`getEscrow`), the outcome selects `confirmDelivery` or `confirmFailure`, and the tracking hash and timestamp are used to derive the report ID stored in `processedReports`.

4. LITERATURE REVIEW

Four peer-reviewed works were selected because each addresses a component that ProofPay depends on: blockchain's role in e-commerce transactions, the oracle problem as a research topic, the legal and semantic difficulties of feeding smart contracts with external facts, and the use of smart contracts to control information quality in supply chains. Bibliographic details were verified against Crossref records for the DOIs cited, and the summaries below paraphrase the published contributions rather than quoting them.

4.1 Review of selected work

[1] Albshaier, Almarri and Hafizur Rahman (2024) review blockchain's role in e-commerce transactions and set out open challenges and future research directions. The review groups the literature by the function blockchain performs — trust and transparency, payment and settlement, product provenance, and identity — and observes that most published e-commerce systems address one function at a time while commercial platforms need several simultaneously. It highlights the persistent difficulty of connecting on-chain logic to the operational realities of retail, including returns, disputes and logistics data, and it identifies integration with existing payment infrastructure and regulatory compliance as unresolved. For ProofPay the review is useful as a map of scope: it confirms that payment protection is a recognised function of blockchain in commerce, and that the weakest link in published designs is not the ledger but the interface between the ledger and the physical shipment.

[2] Caldarelli (2022) provides an overview of blockchain oracle research, organising the field by the problem being solved (authenticity, confidentiality, availability), by the architecture used (centralised, decentralised, consensus-based, hardware-based), and by the guarantees each can offer. The paper is explicit that an oracle is a trust boundary: a contract's correctness is only as strong as the data it receives, and the literature's proposed mitigations trade cost and latency for stronger assumptions. Two ideas from this work shape ProofPay directly. The first is that the oracle should be a separate, replaceable component rather than a privileged function inside the application contract — reflected in the receiver being a distinct contract with a settable forwarder. The second is that provenance matters: a report is more useful when it carries the information needed to identify how it was produced, which is why the workflow emits a second, provenance-bearing record alongside the bytes it submits.

[3] Nguyen, Le, Duong and Li (2023) examine blockchain oracles in relation to smart contracts, focusing on legal reasoning and on how the oracle problem manifests when contracts must be interpreted and applied to real-world facts. The authors argue that the difficulty is not merely transport of data but the translation of an ambiguous external fact into a value the contract can act on unambiguously, and they discuss the evidentiary role of the data source when a dispute later arises. This framing explains a design choice in ProofPay that would otherwise look redundant: the workflow hashes the tracking ID and records the observation timestamp inside the report, and the receiver stores a report ID derived from all four fields. The effect is that a settlement can be tied back to a specific observation of a specific record at a specific time, which is the on-chain analogue of the evidentiary chain the paper describes.

[4] Allenbrand (2023) studies smart-contract-enabled consortium blockchains for controlling information distortion in supply chains. The work addresses a problem adjacent to ProofPay's: participants in a supply chain have incentives to report favourable information, and a shared ledger alone does not fix that unless the write path and its permissions are designed carefully. The proposed approach combines consortium governance with contract-enforced rules about who may record which events and in what order. For ProofPay the relevant lesson is that validation must be layered — the API validates transitions, the workflow re-reads on-chain state before deciding, the receiver re-checks authorisation and replay, and the escrow re-checks funding. No single layer is trusted to be sufficient, which is the practical answer to the distortion problem in a system where the data source is not itself decentralised.

4.2 Summary of the literature

Table 3: Literature Review Summary

Sl.	Author(s) & year	Title / source	Approach	Findings	Limitation / gap
1	Albshaier, Almarri, Hafizur Rahman (2024)	A Review of Blockchain's Role in E-Commerce Transactions: Open Challenges and Future Research Directions. *Computers*, 13(1), 27.	Systematic review of blockchain applications in e-commerce, grouped by function (trust, payment, provenance, identity) with an open-challenges analysis.	Blockchain improves transparency and settlement assurance; published systems usually address one function only; logistics and dispute integration remain the weakest points.	Review-level, not an implemented settlement pipeline; does not provide an oracle design that turns courier status into a contract-executable outcome.
2	Caldarelli (2022)	Overview of Blockchain Oracle Research. *Future Internet*, 14(6), 175.	Taxonomy of oracle architectures and guarantees, organised by authenticity, confidentiality and availability problems.	Oracles are the trust boundary of smart contracts; decentralised and consensus-based designs reduce reliance on a single reporter at the cost of latency and complexity.	Conceptual overview; gives no concrete receipt-verified pipeline linking an HTTP data source to an escrow settlement.
3	Nguyen, Le, Duong, Li (2023)	Blockchain Oracles: Implications for Smart Contracts in Legal Reasoning and Addressing the Oracle Problem. SoICT '23, pp. 296–303.	Analysis of the oracle problem with emphasis on translation of real-world facts into contract-interpretable values and on evidentiary traceability.	The core difficulty is semantic translation and evidence, not transport; provenance of the reported fact is essential to later dispute resolution.	Legal-analytical rather than systems-level; no reference implementation of provenance fields inside an escrow settlement report.
4	Allenbrand (2023)	Smart contract-enabled consortium blockchains for the control of supply chain information distortion. *Blockchain: Research and Applications*, 4(3), 100134.	Consortium design in which governance and contract rules constrain who records which supply-chain events and when.	Shared ledgers alone do not prevent distorted reporting; layered validation and explicit write permissions are required.	Assumes a permissioned consortium; does not address an open payment escrow whose data source is an ordinary web API.

4.3 Research gap and positioning

Read together, the four works describe a system whose custody layer is well understood and whose data layer is not. [1] establishes that payment protection is a legitimate blockchain application and that logistics integration is where published designs stall. [2] explains why the oracle is the trust boundary and what architectural choices exist for strengthening it. [3] sharpens the point: the hard part is not moving a status string from an API into a contract, but converting an operational fact into an unambiguous, evidenced value that a court or an auditor could later interrogate. [4] adds that validation must be distributed across layers, because any single reporter with an incentive can distort what the ledger records.

None of the four supplies what a working payment-protection system needs: a pipeline in which an oracle component fetches a courier record itself, verifies that the corresponding escrow is still funded on chain, applies an explicit and inspectable decision table, encodes a report that carries provenance, and hands it to a receiver contract that enforces authorisation, replay protection and single settlement before instructing a custody contract to pay. That is the specific combination ProofPay implements. It does not claim to invent escrow, ERC-20 tokens or oracles; each of those is mature. What it contributes is an end-to-end, executed and measured instantiation of the combination on a shipment-linked payment, with the decision isolated in the CRE workflow, the enforcement isolated in the contracts, and the two connected by a relay whose lack of decision authority is a documented design property rather than an implementation detail. The system also makes its own trust boundary explicit — local simulation instead of DON consensus, a simulated courier instead of a carrier API — which the literature identifies as the component that most needs to be honestly characterised.

5. EXISTING SYSTEM

Before designing ProofPay it was necessary to establish what already exists for protecting a payment between a buyer and a seller who do not trust each other. Three families of practice are relevant: the payment-protection mechanisms built into large marketplaces, independent commercial escrow services, and blockchain escrows that rely on a manual administrator or oracle. Each solves part of the problem, and each leaves a specific gap that this project addresses. The descriptions below are written at the level of mechanism rather than of any particular vendor's marketing, because the exact commercial terms of individual providers are not public and are not reproduced here.

5.1 Centralised marketplace payment protection

The dominant model is a platform that sits between buyer and seller and holds the money. The buyer pays the platform; the platform informs the seller that the order may be fulfilled; the seller ships; and the platform releases the funds after a holding period, a delivery confirmation, or the expiry of a dispute window. This works at scale because the platform has two powerful instruments: it can reverse a card transaction, and it can exclude a seller from future trade.

The weakness is that the guarantee is a contractual promise attached to a private ledger. The buyer cannot verify that the funds exist, cannot see the rule that will be applied, and cannot prove afterwards which rule was applied. When a dispute is raised, resolution depends on human review of tracking numbers, photographs and chat logs. The delay is measured in days, and the outcome is recorded only inside the operator's database. For a high-value purchase the buyer is therefore trusting an institution rather than a mechanism, and the seller is accepting a settlement delay that ties up working capital.

5.2 Commercial escrow services

Dedicated escrow providers separate the custody function from the marketplace. A neutral third party opens an escrow account, receives the purchase price, and pays the seller when the agreed condition is met, typically a signed acceptance or a courier delivery record. For property, legal settlements and business acquisitions this is a mature and heavily regulated activity.

For ordinary e-commerce the model is economically unattractive: onboarding is manual, fees are proportional to value, and each transaction requires the parties to be identified and accepted. The escrow agent is also a single point of failure and of trust. The condition itself — "was the parcel delivered?" — is evaluated by a human reading a courier's web page, so the same evidentiary weakness as in Section 5.1 reappears. Nothing about the arrangement is programmatically inspectable, and there is no audit trail that an outside party can independently verify.

5.3 Blockchain escrow with a manual oracle

A smart-contract escrow removes the custodian. Funds are held by bytecode, the release condition is expressed in Solidity, and every state change is recorded as an event that anyone can read. The weak point moves elsewhere: a contract cannot observe the physical world. Someone must tell it that the parcel arrived, and that someone is the oracle.

In the simplest and most common form, the oracle is an administrator's wallet: an externally owned account calls `confirmDelivery(orderId)` or `confirmFailure(orderId)` after checking a tracking page. This is a genuine improvement in custody and auditability, but it reintroduces exactly the trust that the blockchain was meant to remove, now concentrated in a single key. The administrator can settle the wrong order, settle twice, or refuse to settle at all, and the only recourse is social. The ProofPay escrow contract deliberately retains this path (`oracle` is a settable address, and the receiver exposes a `manualOracle` role) because it is indispensable for local testing and for recovering from an oracle outage — but it is not the mechanism the project relies on for automation.

5.4 Comparative observations

Table 4 summarises the three approaches against the properties that matter for a shipment-linked purchase: who holds the money, who decides that the condition is satisfied, how quickly the decision is made, what an outside party can verify, and how the design fails. The pattern is consistent — custody and verification are traded against each other. Centralised systems have strong verification resources but opaque custody; on-chain escrows have transparent custody but no native ability to observe delivery.

Table 4: Existing Systems

Sl.	Existing system / method	How it works	Strengths	Limitations
1	Centralised marketplace payment protection	The platform receives the buyer's payment, instructs the seller to fulfil the order, and releases funds after a holding period, a delivery confirmation or a dispute window expires.	Familiar to users, fast at scale, chargeback leverage over sellers, one integration for both parties.	Custody is a private ledger promise; rules are not inspectable; disputes are resolved by human review over days; the audit trail is internal only.
2	Commercial escrow service	A licensed intermediary opens an escrow account, holds the purchase price, and pays the seller once a signed acceptance or delivery record is presented.	Legally robust for large transactions; regulated custody; neutral third party.	Manual onboarding, value-proportional fees, poor fit for small orders; the delivery condition is still checked by a person; single trusted institution.
3	Blockchain escrow with manual oracle	A Solidity contract holds ERC-20 funds and an administrator wallet calls the settlement function after reading a courier tracking page.	Transparent custody; deterministic rules; public event log; no intermediary holding the money.	Verification collapses to one key; no independent evidence trail for the decision; operator must be online; duplicate or wrong settlements are possible if the caller is careless.

The gap that motivates ProofPay is the third row. Custody is already solved by smart contracts; what is missing is a credible, auditable and reproducible path from an off-chain shipment record to an on-chain settlement, with the decision made by code that can fetch data, cross-check the contract state, and produce a signed artefact describing what it decided and why. That is precisely the role of a Chainlink CRE workflow, and it is the component this project builds and demonstrates.

6. PROPOSED SYSTEM

6.1 System overview

ProofPay is a three-tier decentralised application. A React front end talks to the user's MetaMask wallet and to a small Express service; the wallet talks to a local Hardhat blockchain on which three contracts are deployed; and a Chainlink CRE workflow runs beside them, reading the Express service over HTTP and the escrow contract over EVM, deciding whether an order is eligible for settlement, and emitting a settlement report. A local execution bridge carries that report to the receiver contract, which is the only contract authorised to instruct the escrow.

The payment asset is ProofPay USD (pUSD), an ERC-20 token with eighteen decimals and an owner-restricted `mint` function. A buyer approves the escrow contract for the order amount and calls `createEscrow(orderId, seller, amount)`; the contract pulls the tokens with `safeTransferFrom` and records an `Escrow` struct in the `Funded` state. From that moment the buyer's tokens are held by bytecode. There is no withdrawal function, no owner override and no upgrade path: the only two functions that can move the money are `confirmDelivery`, which pays the seller, and `confirmFailure`, which repays the buyer, and both are guarded by the `onlyOracle` modifier.

Independently of the chain, the seller registers the shipment through the Express API, which persists a record containing the order ID, a tracking ID, a carrier label, a status and an append-only event history. A demo courier scheduler can advance that status automatically. The CRE workflow then performs the cross-check that neither side can perform alone: it fetches the shipment list from the API, reads the corresponding escrow from the chain, and only if the escrow is still `Funded` does it turn the shipment status into a settlement outcome. The outcome is ABI-encoded as `(bytes32 orderId, string outcome, bytes32 trackingIdHash, string observedAt)` and emitted as a structured log line. The bridge relays those exact bytes to `ProofPayCREReceiver.processReport(metadata, report)`, which re-checks authorisation, replay and funding before calling the escrow.

6.2 Actors and responsibilities

Buyer. Connects a wallet, selects a product, approves pUSD and funds the escrow. The buyer's dashboard lists every escrow in which the connected address is the buyer, with live status, amount, settlement transaction and shipment events. The buyer never needs to ask the seller or the platform for a refund: if the shipment is recorded as lost, the refund is produced by the same pipeline that pays sellers.

Seller. Sees incoming escrows where the connected address is the seller, the total value held for them, and the settlements already paid. The seller registers the shipment and tracking ID, which is what makes the order visible to the oracle at all. Funds cannot be withdrawn early, so the seller's incentive is to ship and to have the delivery recorded.

Administrator / oracle operator. Owns the contracts, sets the escrow's oracle address, and is the receiver's `manualOracle`. In the current deployment the forwarder is the zero address, so the manual oracle path is the one exercised. The admin dashboard exposes the read-only verification of the on-chain wiring, the shipment simulator, and the manual settlement panel used before the CRE pipeline was introduced.

Courier data source. The Express shipment API is the off-chain system of record for physical progress. It is deliberately a separate service with its own validation rules, because the whole point of the oracle is to read a source that the contract cannot see. In this project the courier is simulated; the interface, however, is a normal HTTP JSON API, which is what a real carrier integration would also expose.

CRE workflow. The decision-maker. It holds no keys, cannot move funds and is not trusted by the contracts; it produces a report that the receiver independently validates.

Local execution bridge. The relay. It runs the workflow, parses the structured report, validates it against the chain, and submits exactly one transaction per eligible report. It contains no settlement rules of its own.

Smart contracts. The enforcement layer. `ProofPayUSD` is the asset, `ProofPayEscrow` is the custodian and settlement engine, and `ProofPayCREReceiver` is the authorisation and anti-replay boundary in front of the escrow.

6.3 Settlement decision logic

The decision is a pure function of the shipment status, implemented once in the workflow as `shouldSettle`. `DELIVERED` maps to `RELEASE` with outcome `DELIVERED`; `LOST` maps to `REFUND` with outcome `LOST`; `CREATED`, `SHIPPED` and `IN_TRANSIT` map to `NONE`, leaving the escrow funded; `RETURNED` maps to `MANUAL_REVIEW`, which deliberately produces no report because a return after delivery is a dispute, not an automatic outcome; and any unrecognised status maps to `NONE` with a logged reason. The bridge repeats none of this logic. It accepts only the two outcomes the receiver's ABI supports and refuses anything else, but it never decides which outcome applies.

6.4 Why a dedicated pUSD token

Accepting multiple cryptocurrencies would multiply the escrow's accounting surface: each asset has its own decimals, its own transfer semantics and its own failure modes, and the settlement logic would have to branch on token behaviour. A single ERC-20 keeps the contract small, makes amounts directly comparable in tests, and lets the token be minted on demand in a local environment. pUSD is explicitly a prototype: it is not a stablecoin, has no reserve, no peg mechanism and no regulatory status. Its purpose is to represent a fiat-denominated claim inside the demonstration so that gas, which is paid in test ETH, is cleanly separated from the value being escrowed.

6.5 The local execution bridge and what it is not

Chainlink CRE workflows are executed by a DON, and on-chain writes are routed to chains identified by a CRE chain selector. The installed SDK (version 1.23.0) exposes testnet and mainnet selectors but no selector for a local Hardhat node on chain ID 31337, and the environment used for this project has no access to a deployed DON. The workflow therefore runs through the CRE CLI simulator locally, with `dryRun` kept `true` so that its `writeReport` call is never attempted.

That leaves a genuine report with nowhere to go. The bridge exists to close that gap: it launches the same simulation the developer would run by hand, captures the report the workflow produced, and submits those bytes from an unlocked local account. The distinction is important and is stated plainly throughout this report: this is *actual CRE workflow execution* — the HTTP fetch, the EVM read, the decision and the ABI encoding are all performed by CRE — combined with a *local relay* that replaces DON consensus and DON signing. It is not a decentralised oracle network, and the trust model is correspondingly weaker: the relay machine could suppress or forge a report, and only the receiver's authorisation and replay checks stand in the way. Those checks, not the bridge, are what protect the funds.

6.6 Improvements over the existing approaches

Compared with a marketplace hold, ProofPay moves custody into code that the buyer can inspect, and records every settlement as an event with a transaction hash, a block number and a report identifier. Compared with a commercial escrow, it removes the intermediary and the per-transaction onboarding, and it makes the release condition machine-readable. Compared with a bare on-chain escrow driven by an administrator, it separates the decision from the key: the workflow fetches its own evidence, re-reads the escrow state before deciding, and emits a structured record of its reasoning; the receiver then accepts the report only from an authorised caller, only once, and only for an order that is still funded. The administrator key can still settle manually, but it is no longer the only path, and the automated path leaves a far richer audit trail than a bare function call.

7. TECHNOLOGIES USED

Every version below was read from the manifests in the repository rather than assumed: `contracts/package.json`, `frontend/package.json`, `backend/package.json`, `chainlink/cre/shipment-workflow/package.json` and the Solidity settings in `contracts/hardhat.config.js`. The CRE CLI reports itself as version 1.39.0 and the SDK dependency is declared as `^1.23.0`.

Technology stack as installed in the ProofPay repository

Technology	Version	Purpose in ProofPay
Solidity	0.8.28	Language of the three contracts; compiled with the optimizer enabled at 200 runs.
Hardhat	2.29.1	Compilation, testing and the local JSON-RPC node on chain ID 31337; the deployment target for every contract in this report.
Hardhat Network Helpers / Chai Matchers	1.1.0 / 2.1.0	Fixture-based test setup and custom-error assertions for the 31 contract tests.
ethers.js	6.17.0	Deployment scripts, contract tests, the bridge's transaction submission and the read-only evidence scripts.
OpenZeppelin Contracts	5.6.1	`ERC20`, `Ownable`, `ReentrancyGuard`, `SafeERC20` and `IERC165`.
Node.js	22.23.2 (runtime)	Runtime for the Express API, the Hardhat toolchain and all TypeScript utilities.
Express	4.21.2	Shipment API server on port 5001, with CORS and JSON body limits.
React	19.3.1	Single-page application for the buyer, seller and admin experiences.
TypeScript	5.7.2	Type checking across the front end and the CRE workflow sources.
Vite	8.3.5	Development server on port 5173 and the production bundle.
Tailwind CSS	4.3.3	Utility-first styling through the official Vite plugin; no component library is used.
React Router	7.18.5	Client-side routes for the dashboards, checkout and order tracking.
viem	2.57.14	Hashing, ABI encoding and decoding in the CRE workflow; also used in the front end's typed contract calls.
MetaMask	Browser extension	EIP-1193 provider: account access, chain switching, `eth_call`, `eth_getLogs` and `eth_sendTransaction`.
Chainlink CRE SDK	^1.23.0	`CronCapability`, `HTTPClient`, `EVMClient`, `runtime.report` and the consensus helpers.
Chainlink CRE CLI	1.39.0	`cre workflow simulate`, which executes the workflow locally against the staging configuration.
JSON file persistence	Application-level	Single-file shipment store with atomic replacement, read and written by the Express service.
Git / GitHub	—	Version control; no public repository URL is configured in the project metadata.

Three points about the environment deserve emphasis because they are frequently misreported for projects of this kind. First, the blockchain is **Hardhat Network** running as a local node; the project contains no Ganache, Remix, Sepolia or mainnet deployment, and the only chain ever contacted during the work described here is `http://127.0.0.1:8545` with chain ID 31337. Second, the external data source is the project's own Express service, not a carrier API. Third, gas is paid in test ETH from Hardhat's deterministic accounts while the value under escrow is pUSD; the two are never mixed.

8. ARCHITECTURE DIAGRAM

Figure 1 presents the deployed architecture as a layered diagram. Reading from the top, the user layer contains the three human roles and the simulated courier. The presentation layer is the React application together with the MetaMask provider that mediates every chain interaction. The backend layer is the Express API, its validation service and the JSON store. The Chainlink layer is the CRE workflow with its three capabilities. The local execution layer is the report export and the bridge. The blockchain layer is the Hardhat node with the three contracts. The two settlement outcomes appear at the bottom, because they are both the end of the flow and the reason the flow exists.

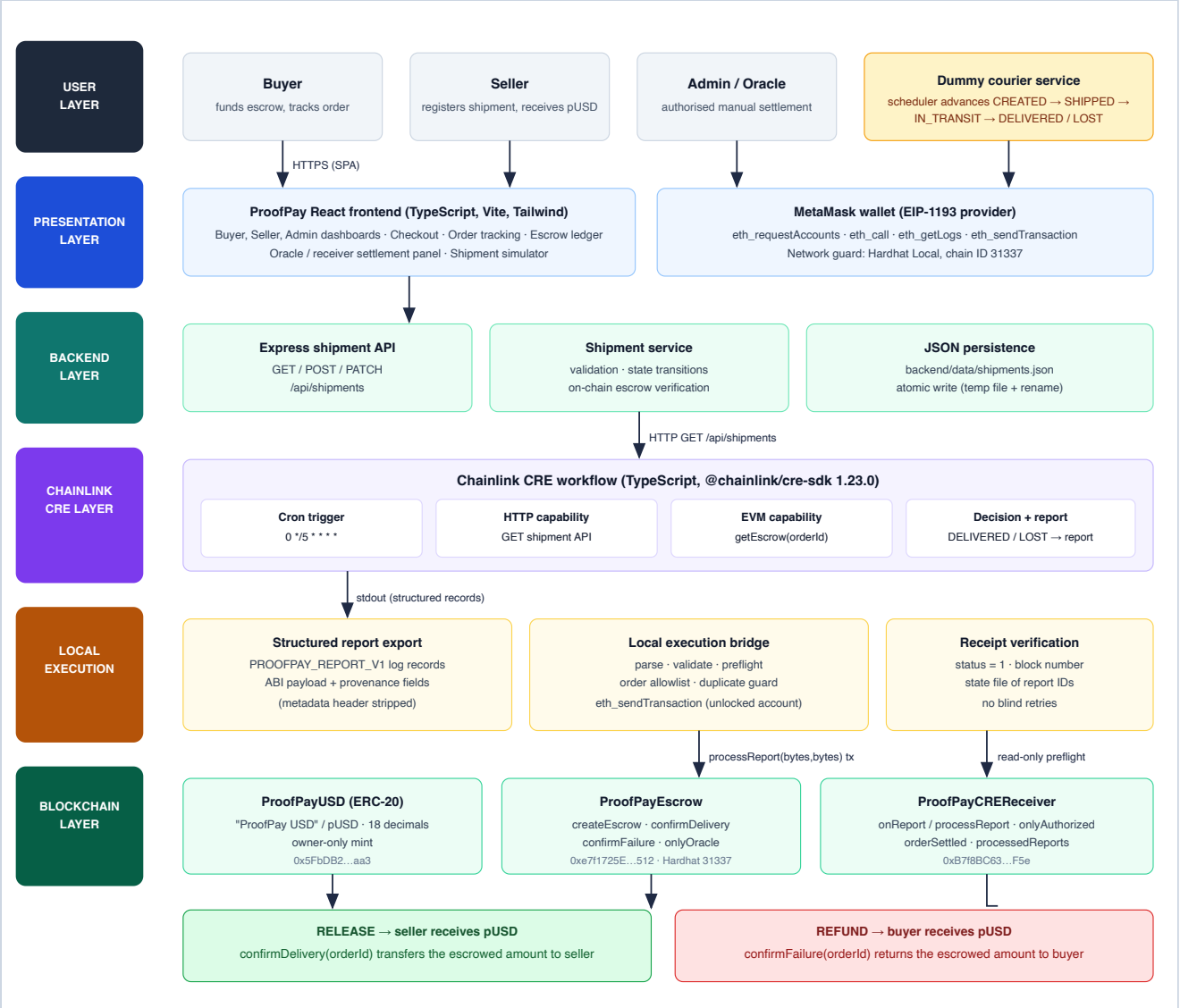


Figure 1: Overall Architecture of the Proposed ProofPay System

User layer. The buyer, seller and admin are ordinary browser users; the only software they need is MetaMask and a browser. The courier box is drawn in the same layer because it is an independent actor, not a part of ProofPay: the scheduler in `courier-scheduler.ts` plays the role that a carrier's tracking system would play in production.

Presentation layer. The React application never holds a key. It reads state with `eth_call` and `eth_getLogs`, discovers escrow order IDs from `EscrowCreated` logs, and then re-reads each escrow through `getEscrow` so that the contract remains the single source of truth. Writes are requested through the provider and signed inside the wallet; the network is verified to be chain ID 31337 before any write is offered.

Backend layer. The Express service exposes `GET /api/shipments`, `GET /api/shipments/:orderId`, `POST /api/shipments` and `PATCH /api/shipments/:orderId/status`. Validation, allowed transitions and event-his-

tory append live in `shipmentService.js`; persistence is a single JSON document written atomically. Mutating routes require a shared secret header, which is what allows the courier scheduler to act as a distinct client rather than as the browser.

Chainlink layer. The workflow is triggered by a cron expression, fetches the shipment list over HTTP, reads each escrow over EVM at the last finalised block, applies `shouldSettle`, and encodes the report. Because the report must survive to the dry-run branch, generation happens before the `dryRun` check — a sequencing detail that had to be corrected during development and that is discussed in Section 10.7.

Local execution layer. The workflow logs two tagged records per eligible order: one carrying the exact bytes for `processReport`, and one carrying provenance (raw report length, metadata header length, hash, configured receiver and escrow, and whether dry-run was active). The bridge joins them by order ID, filters against an allowlist, verifies the chain wiring, and submits.

Blockchain layer. The token, escrow and receiver are deployed at fixed addresses on the local node. The escrow's oracle is the receiver; the receiver's forwarder is the zero address and its manual oracle is the admin account. That combination is what makes the local demonstration possible: reports arrive through `processReport` from the admin account, are accepted as manual-oracle calls, and are then executed by the escrow because the caller of the escrow is the receiver.

Figure 2 shows the same system as a process rather than as a structure, which is the view a reader needs in order to see where the decision is actually taken. Steps 1 to 4 are ordinary application behaviour. Step 5 and step 6 are the oracle: an HTTP read of an off-chain record and an EVM read of on-chain custody, combined into a decision. Steps 7 to 9 are the relay and the contracts, and they are strictly downstream of the decision — no component after step 6 can change the outcome, only refuse to act on it.

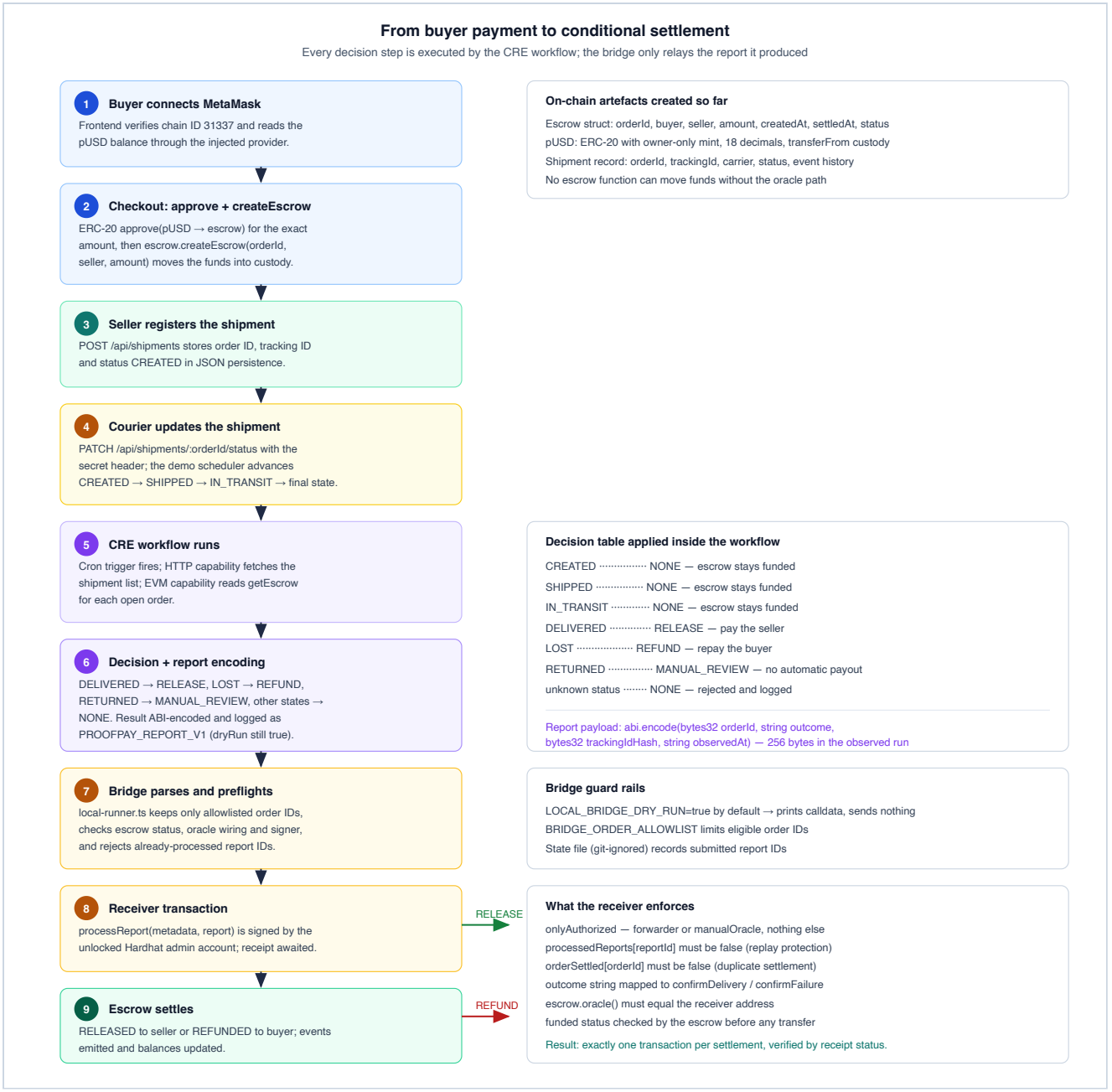


Figure 2: End-to-End Shipment Verification and Settlement Workflow

9. FEATURES

The features below are described from the implementation, and each names the contract function or source file that provides it. Anything that exists only as a plan is marked as such rather than listed as a feature.

9.1 ERC-20 payment token

`ProofPayUSD` is eighteen lines of Solidity over OpenZeppelin's `ERC20` and `Ownable`. The name is "ProofPay USD", the symbol is "pUSD", the decimals are 18, and the initial supply is zero — tokens come into existence only through `mint`, which is restricted to the owner by `onlyOwner`. Minting to the zero address is rejected by OpenZeppelin's `ERC20InvalidReceiver`, and the constructor rejects a zero owner. In the local demonstration the admin minted 1,000,000 pUSD and distributed working balances to the buyer and seller accounts; the total supply at block 32 was 1,010,100 pUSD, the extra 10,100 coming from later top-ups used to seed demonstration escrows.

9.2 Escrow creation and funding

Funding is a two-transaction sequence that users see as one button press: `approve(escrow, amount)` followed by `createEscrow(orderId, seller, amount)`. The contract validates that the order ID is not zero, that the buyer argument equals the caller, that neither party is the zero address, that buyer and seller differ, that the amount is non-zero, and that the order ID has never been used. Only then does it write the struct and pull the tokens. The order ID is generated client-side as a random 32-byte value, so the buyer cannot collide with an existing escrow by accident, and the `EscrowAlreadyExists` error is the backstop if they try.

9.3 Secure fund custody

Once `createEscrow` returns, the tokens belong to the contract. There is no `withdraw`, no `sweep`, no pause and no upgrade proxy. The owner can change the oracle address but cannot move funds; the oracle can settle but only from `Funded` to `Released` or `Refunded`, and only once. The token balance held by the escrow is therefore always the sum of the amounts of the currently funded orders, which is a property that can be checked from outside with two RPC calls.

9.4 Buyer dashboard

The buyer view shows the wallet's pUSD balance and native balance, every escrow in which the connected address is the buyer, and a derived activity feed built from escrow timestamps: "Escrow funded" at `createdAt`, then "Escrow released" or "Escrow refunded" at `settledAt`. Each row links to an order page that shows the escrow ledger, the on-chain record read through `getEscrow`, the settlement transaction, and the shipment tracking panel. Because the order list is discovered from logs and then re-read from storage, a page refresh always shows chain state rather than a cached client view.

9.5 Seller dashboard

The seller view mirrors the buyer view from the other side: incoming escrows, the total currently held in escrow for that address, the total already settled, and a count of refunded orders. From here the seller registers a shipment against a funded escrow, which is the action that makes the order eligible for automated settlement. The dashboard states explicitly that funds are released by the escrow contract only, and that shipment tracking is off-chain data which does not itself move money.

9.6 Admin dashboard

The admin view exposes the operational controls that a demonstration needs: the escrow/oracle wiring read directly from both contracts, the shipment simulator for manual status changes, the manual settlement panel, and the CRE status panel that reports whether the receiver is configured and which oracle paths are live. It is deliberately read-heavy; nothing in it can bypass the escrow's own checks.

9.7 Shipment tracking

Shipment records carry an order ID, a tracking ID, a carrier label, a status, an ISO timestamp and an ordered event list. The seven statuses are `CREATED`, `SHIPPED`, `IN_TRANSIT`, `DELIVERED`, `LOST`, `RETURNED` and — at the store level — a not-found state. Transitions are validated: forward progress is allowed, a final state is terminal, and an unknown transition is rejected with a 409 rather than silently accepted. Each accepted change appends an event with a human-readable note, and the UI renders that history as a timeline.

9.8 Chainlink CRE integration

The workflow uses two of CRE's capabilities for real. `HTTPClient.sendRequest` fetches the shipment list, wrapped in `consensusIdenticalAggregation` so that the response is agreed across nodes in a DON setting; the local simulator executes the same code path with a single node. `EVMClient.callContract` performs an `eth_call` of `getEscrow` at `LAST_FINALIZED_BLOCK_NUMBER`, and the result is decoded with viem's `decodeFunctionResult`. Both capabilities are exercised on every run; the logs in Section 11 show the resulting report being produced for real shipments.

9.9 Automated settlement reports

A report is built by `createReportPayload`, which pads the order ID to 32 bytes, hashes the tracking ID with `keccak256`, and ABI-encodes the four-field tuple. The bytes are then passed through `prepareReportRequest` and `runtime.report`. The SDK prefixes a 109-byte metadata header to the encoded body, and the receiver decodes the body only, so the workflow verifies which slice actually decodes to the expected order and outcome and reports that provenance rather than assuming it. In the observed runs the body was 256 bytes and the raw report 365 bytes.

9.10 Receiver contract

`ProofPayCREReceiver` implements the CRE receiver interface: `onReport(bytes metadata, bytes report)` for the forwarder path, and `processReport(bytes metadata, bytes report)` for the local path. Both funnel into one internal routine. It also implements `supportsInterface` for `IReceiver` and `IERC165`, exposes `setForwarder` and `setManualOracle` to the owner, and keeps the two anti-replay mappings described below.

9.11 Automatic release and refund

On `DELIVERED` the receiver calls `escrow.confirmDelivery(orderId)`, which pays the seller; on `LOST` it calls `confirmFailure(orderId)`, which repays the buyer. Because the receiver is the escrow's oracle, those calls satisfy `onlyOracle` without any privileged key being involved at the escrow layer. Any other outcome string reverts with `UnsupportedOutcome`, which is why the workflow never emits `MANUAL_REVIEW` as a settlement report.

9.12 Access control

Three roles exist. `owner` (OpenZeppelin `Ownable`) may set the escrow's oracle, the receiver's forwarder and the receiver's manual oracle, and may mint pUSD. `oracle` on the escrow is the receiver address, and only that address may settle. On the receiver, `onlyAuthorized` accepts the forwarder, the manual oracle, or the owner when no forwarder is configured; everything else reverts with `UnauthorizedCaller`. The three are distinct on purpose: the key that configures the system is not the key that settles it, and neither can move tokens directly.

9.13 Event logging

Every state change emits an event. The token emits the standard `Transfer` and `Approval`. The escrow emits `EscrowCreated`, `EscrowReleased`, `EscrowRefunded` and `OracleUpdated`. The receiver emits `ForwarderUpdated`, `ManualOracleUpdated`, `ReportProcessed` (order ID, outcome string and report ID, all searchable) and `SettlementExecuted` (order ID and a boolean release flag). The report ID in the event is the same

value stored in `processedReports`, so an auditor can take an event, prove that the report was recorded, and confirm that no second settlement for that order exists.

9.14 Replay and duplicate protection

Two mappings, both confirmed in the source, close the two distinct attack paths. `processedReports[reportId]` is keyed on `keccak256(orderId, outcome, trackingIdHash, observedAt, metadata)`, so resubmitting the same bytes reverts with `ReportAlreadyProcessed`. `orderSettled[orderId]` blocks a different report for an order that has already been paid, reverting with `OrderAlreadySettled`. Beneath both, the escrow independently refuses to settle anything that is not `Funded`, so a bug in the receiver still cannot drain a settled order. The bridge adds a third layer on the client side by refusing to submit a report ID it has already recorded as successful.

10. IMPLEMENTATION

10.1 Requirement analysis and design

Design began from four questions, each of which produced a constraint that shows up directly in the code. Who holds the money during fulfilment? The contract, and only the contract, which rules out any withdrawal or sweep function. Who is allowed to say that fulfilment happened? An oracle component, not a user role, which rules out letting the buyer or seller settle. What evidence exists for that statement? A shipment record plus the escrow's own state, which is why the workflow reads both. What happens if the evidence is wrong or repeated? The receiver rejects repeats and the escrow rejects anything not funded, which is why three independent checks exist instead of one.

The functional requirements that followed were: fund an escrow with an exact amount; register a shipment against a funded escrow; advance a shipment through the courier vocabulary; fetch shipments from an off-chain API inside the oracle component; read the escrow state on chain inside the same component; map status to outcome deterministically; encode a report the receiver can decode; authorise the report's caller; reject repeated reports and repeat settlements; pay the seller on delivery and refund the buyer on loss; and present all of this to three roles without any of them needing a terminal.

The transaction lifecycle is linear and irreversible after settlement: order chosen → allowance granted → escrow funded → shipment registered → shipment advanced → CRE evaluation → report emitted → report relayed → receiver validation → escrow payout. Two properties were treated as non-negotiable during design. Settlement must be idempotent, so that re-running the oracle cannot pay twice. And the oracle must be replaceable, so that the local relay can be swapped for a deployed DON without touching the escrow; that is the reason the receiver holds a settable `forwarder` and the escrow holds a settable `oracle` rather than hard-coding addresses.

The security requirements were derived from the threat model rather than from a checklist: no privileged path to move escrowed funds; no settlement of an order that is not funded; no second settlement of the same order; no acceptance of a report from an unauthorised caller; no acceptance of the same report twice; no decision logic in the relay; and no secret in the browser. Section 12 assesses each of these against the implementation.

10.2 Smart contract development

ProofPayUSD. The token is intentionally minimal: it inherits OpenZeppelin's `ERC20` and `Ownable`, fixes the name, symbol and eighteen decimals in the constructor, and adds a single function, `mint(address,uint256)`, guarded by `onlyOwner`. There is no burn, no pause, no permit and no fee logic, because none of those is needed to demonstrate conditional custody and each would enlarge the audit surface. The constructor rejects a zero owner through OpenZeppelin's `OwnableInvalidOwner`, and minting to the zero address reverts with `ERC20InvalidReceiver`; both behaviours are covered by tests.

ProofPayEscrow. The escrow is the core of the system. Its state consists of an immutable `paymentToken`, a mutable `oracle`, and a private mapping `_escrows` from `bytes32` order ID to an `Escrow` struct holding the order ID, buyer, seller, amount, `createdAt`, `settledAt` and an `EscrowStatus` enum with the values `None`, `Funded`, `Released` and `Refunded`. Four functions change state or read it: `setOracle` (owner only), `createEscrow`, `confirmDelivery`, `confirmFailure` and `getEscrow`.

`createEscrow` validates the order ID, requires that the buyer argument equals `msg.sender`, rejects a zero or identical seller, rejects a zero amount and rejects a reused order ID; it then writes the struct with status `Funded` and pulls the tokens with `safeTransferFrom`. The order of those two steps is deliberate: state is written before the external call, and the function is `nonReentrant`. `confirmDelivery` and `confirmFailure` are both `onlyOracle` and `nonReentrant`, both call the private `_getFundedEscrow` helper, and both set the terminal status and `settledAt` before transferring. `_getFundedEscrow` reverts with `EscrowNotFound` for an unknown order and `EscrowNotFunded` for anything that is not `Funded`, which is what makes double settlement impossible at the custody layer regardless of what calls it.

The contract emits `EscrowCreated`, `EscrowReleased`, `EscrowRefunded` and `OracleUpdated`, and defines ten custom errors, all of them distinct: `InvalidToken`, `InvalidOrderId`, `InvalidBuyer`, `InvalidSeller`, `SameBuyerAndSeller`, `InvalidAmount`, `InvalidOracle`, `EscrowAlreadyExists`, `EscrowNotFound`, `EscrowNotFunded` and `UnauthorizedOracle`. Custom errors were preferred to require strings because they are cheaper and because the tests can assert on the exact error and its arguments.

ProofPayCREReceiver. The receiver is the authorisation and anti-replay boundary. It stores an immutable `IEscrowSettlement escrow`, a mutable `forwarder`, a mutable `manualOracle`, and two mappings: `processedReports` keyed by report ID and `orderSettled` keyed by order ID. It implements `IReceiver` and `IERC165` so that a deployed DON can verify the interface, and it exposes both `onReport` (the CRE entry point) and `processReport` (the local relay's entry point); both are `onlyAuthorized` and both delegate to one internal `_processReport` so that the two paths cannot diverge.

`_processReport` decodes `(bytes32 orderId, string outcome, bytes32 trackingIdHash, string observedAt)`, rejects a zero order ID, derives `reportId = keccak256(orderId, outcome, trackingIdHash, observedAt, metadata)`, rejects a report ID already recorded, rejects an order already settled, and asks the escrow whether the order is still funded through an external self-call wrapped in `try/catch` so that a missing escrow surfaces as `EscrowNotFound` rather than as a raw revert. It then compares `keccak256(bytes(outcome))` against the two supported strings and calls `confirmDelivery` or `confirmFailure`, marks both mappings, and emits `ReportProcessed` and `SettlementExecuted`. Any other outcome reverts with `UnsupportedOutcome`. The design point is that the receiver contains no notion of "delivered" or "lost" beyond the two strings it is willing to forward: it is a policy boundary, not a decision engine.

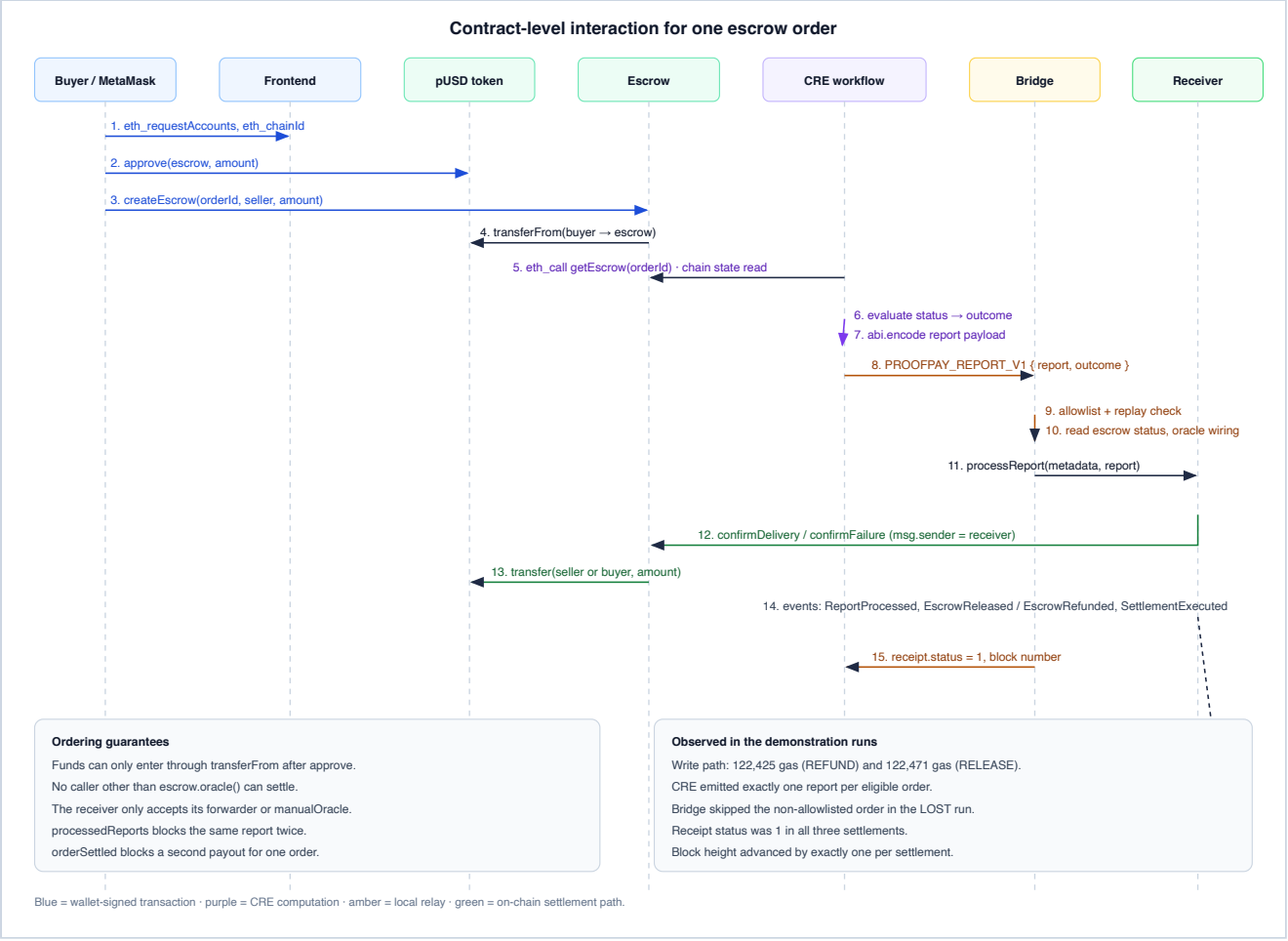


Figure 3: Contract Interaction Sequence for a Single Escrow Order

10.3 Compilation and testing

Compilation is driven by Hardhat 2.29.1 with Solidity 0.8.28 and the optimizer enabled at 200 runs, configured in `contracts/hardhat.config.js`. Three networks are declared: an in-process `hardhat` network on chain ID 31337, a `localhost` network pointing at `http://127.0.0.1:8545` with a 16,000,000 gas ceiling, and a `sepolia` entry that is present but unused — no deployment in this project ever targeted it, and its presence is configuration only.

The tests use Mocha through Hardhat, Chai for assertions, `@nomicfoundation/hardhat-chai-matchers` for event and custom-error assertions, and `loadFixture` from the network helpers so that each case starts from a deterministic snapshot. The suite is organised by contract: 6 tests for the token, 16 for the escrow and 9 for the receiver. Positive cases check the happy path and the emitted events; negative cases check that unauthorised callers, duplicate funding, duplicate settlement, unknown orders, unfunded orders, reentrant creation and invalid deployment arguments all revert with the specific custom error and, where relevant, the specific arguments. The full result is reported in Section 11.1.

10.4 Local blockchain deployment

The node is started with `npx hardhat node` from the `contracts` directory. It listens on `127.0.0.1:8545`, reports chain ID 31337 and exposes twenty deterministic accounts funded with 10,000 test ETH each. Deployment order matters and is fixed: the token first, because the escrow's constructor requires a token address that already has bytecode; the escrow second, with the deployer as owner; the receiver third, with the escrow address as its settlement target; then `escrow.setOracle(receiver)` so that the receiver becomes the only address allowed to settle; and finally the receiver's `manualOracle` is set to the admin account while `forwarder` is left at the zero address, which is what selects the manual path for local execution.

`scripts/deploy.js` performs the token and escrow deployment, `scripts/deploy-receiver.js` deploys the receiver and writes `contracts/deployments/cre-receiver-localhost.json` recording the network, chain ID, deployer, owner, escrow, forwarder, manual oracle and receiver addresses with a timestamp, and `scripts/authorize-receiver.js` applies the two authorisation calls and verifies them by reading back `escrow.oracle()` and `receiver.manualOracle()`. MetaMask is pointed at the same node through a custom network entry with chain ID 31337 and the node's RPC URL; the front end refuses to offer write actions until the wallet reports that chain ID.

10.5 Frontend development

The front end is a React 19 single-page application built with Vite 8 and styled with Tailwind CSS 4 through the Vite plugin. Routing is handled by React Router 7 across five views: landing, buyer dashboard, seller dashboard, checkout and order tracking, plus the admin dashboard. State is not kept in a global store of business facts; instead two hooks derive it from the chain. `useWallet` owns the EIP-1193 connection — it rehydrates silently on mount with `eth_accounts`, subscribes to `accountsChanged` and `chainChanged`, reads the native balance, and exposes `connect`, `disconnect` and `switchToHardhatLocal`, the last of which handles error 4902 by offering to add the network. `useEscrows` discovers order IDs from `EscrowCreated` logs and then reads every field back through `getEscrow`, so the contract remains the source of truth even if a log is missed.

Writes are modelled as explicit transaction states rather than fire-and-forget calls. The checkout page performs the allowance check, sends `approve`, waits for the receipt, sends `createEscrow`, waits again, and only then navigates to the order page; each step surfaces pending, confirmed and failed states, and a rejected wallet prompt is reported as a rejection rather than as a chain error. The receiver path is isolated in `lib/receiver.ts`, which encodes the report with viem and sends `processReport` from whichever account the wallet exposes — the mechanism used by the admin panel before the automated bridge existed. That panel remains functional and is independent of the CRE pipeline.

10.6 Backend development

The backend is an Express 4 service on port 5001. `app.js` installs CORS restricted to the configured front-end origin, a 32 kB JSON body limit, the shipment router at `/api/shipments`, a 404 handler and an error handler that maps validation failures to 400, missing records to 404 and illegal transitions to 409. The router exposes the collection read, the

single-record read, an events read, registration and the status patch. Mutating routes are protected by a shared-secret middleware, so the scheduler and a browser client are distinguishable.

`shipmentService.js` holds the rules. Registration requires a 32-byte order ID, a non-empty tracking ID and a carrier label, and refuses a second record for the same order. A status change must be one of the seven known values and must be a legal transition from the current one; a terminal state accepts no further change, and an identical status is treated as a no-op rather than an error. Every accepted change appends an event with an ISO timestamp and a short human-readable note, which is what the tracking timeline in the UI renders. `shipmentStore.js` persists the whole collection as a single JSON document, writing to a temporary file and renaming it into place so that a crash mid-write cannot truncate the store, and serialising writes through a promise chain so that two concurrent requests cannot interleave.

The service also verifies escrow state when a shipment is registered: it reads the escrow through a minimal ABI and rejects a shipment whose order is not funded, which prevents the oracle from ever being asked about an order that has no money behind it. This is a convenience check, not a security boundary — the receiver performs its own funding check at settlement time — but it catches the common mistake early, in the UI, where it is cheap to fix.

10.7 Chainlink CRE workflow development

The workflow lives in `chainlink/cre` with a `project.yaml` declaring the project and a `workflow.yaml` declaring the target, and the implementation in `shipment-workflow/`. Configuration is typed in `config.ts` with zod validation and supplied per target by `config.staging.json` and `config.production.json`; the staging file is the only one used, and the production file was left untouched throughout the work described here. The staging configuration names the API base URL, the API path, the RPC URL, chain ID 31337, the escrow and receiver addresses, the forwarder address (zero, meaning the manual path), and `dryRun: true`.

`main.ts` registers a single handler on a `CronCapability` trigger with the expression `0 */5 * * * *`. On each trigger it fetches the shipment collection with `HTTPClient.sendRequest` wrapped in `consensusIdenticalAggregation`, so that in a DON deployment the response body must agree across nodes before it is used; it parses the body defensively, accepting either a bare array or an object with a `shipments` property; it selects the network with `getNetwork` and constructs an `EVMClient`; and it iterates the shipments.

For each shipment, `shouldSettle` is applied first, and shipments that resolve to `NONE` or `MANUAL_REVIEW` are logged and skipped without any chain access, which keeps the read volume proportional to the work that actually exists. For the remaining ones the workflow encodes a `getEscrow` call with viem, sends it through `callContract` at `LAST_FINALIZED_BLOCK_NUMBER`, decodes the result, and skips the shipment unless the escrow reports status 1 (`Funded`). This is the second, independent guard against settling a closed order: the first is the API's own state, and the third will be the receiver's.

Report construction comes next, and the ordering here is the single most important implementation detail in the workflow. The original code checked `config.dryRun` before calling `runtime.report`, which meant that with dry-run enabled — the setting required for safe local work — no report object was ever produced and therefore nothing could be captured. The corrected order is: build the payload, call `prepareReportRequest`, call `runtime.report`, extract and verify the exact bytes, emit the structured records, and only then branch on `dryRun` to decide whether `writeReport` is attempted. The report is therefore real in both modes; in dry-run it simply has no on-chain destination.

Extraction deserves its own paragraph because it is where a subtle bug would silently corrupt settlements. The SDK returns an object whose `rawReport()` is the encoded body prefixed with a 109-byte metadata header (`REPORT_METADATA_HEADER_LENGTH`), and whose `body()` is the payload alone. The receiver decodes the bytes it is given, so the header must be stripped. Rather than assume a fixed offset, `extractAbiPayload` attempts to decode both candidates and returns the one that yields the expected order ID and outcome, reporting which source was used; if neither decodes it returns the raw bytes marked `unverified` and logs that the bridge must not submit them. In the

observed runs the source was `body`, the payload was 256 bytes, the raw report was 365 bytes, and the 109-byte difference matches the header length exactly.

Two log records are then emitted, both prefixed `PROOFPAY_REPORT_V1:`. The first has `type: "report"` and carries the order ID, outcome, tracking ID, observation time, metadata and the payload hex — these are the bytes that will be submitted. The second has `type: "meta"` and carries provenance: the action, the tracking ID hash, which slice the payload came from, whether it decodes as the receiver ABI, the raw report length and header length, the raw report hash, the configured receiver and escrow, and the value of `dryRun`. The split exists because the CRE runtime truncates user log lines at roughly one kilobyte; separating provenance from payload keeps the hex intact, and the bridge rejoins the two records by order ID.

10.8 Local execution bridge

`local-runner.ts` is the only component that submits a report outside CRE. It is deliberately narrow. It loads a typed configuration from environment variables — RPC URL, expected chain ID, receiver, escrow, expected signer, allowlist, dry-run flag, simulation timeout, state file and CRE command — then acquires a lock file so two invocations cannot overlap, then runs the preflight, then executes the CRE simulation as a child process, then parses, validates and submits.

The preflight is a read-only sequence of checks that must all pass before any transaction is even prepared: the RPC answers with the expected chain ID (31337); the receiver address has bytecode; the escrow address has bytecode; the signer account is unlocked on the node, proved by asking the node for its accounts; `escrow.oracle()` equals the receiver; `receiver.manualOracle()` equals the signer; and the signer holds native balance for gas. Because the signer is an account already unlocked by the local node, no private key is ever read, stored or printed — the bridge sends `eth_sendTransaction` and the node signs. That is a deliberate reduction of the secret-handling surface: a private key in an environment file would have been the single most dangerous artefact in the repository.

The simulation is started with the same command a developer would type — `cre workflow simulate shipment-workflow --target staging-settings --trigger-index 0 --non-interactive` — using `spawn` rather than a shell so that arguments are not re-parsed, with a timeout and a captured stdout/stderr. Output is consumed line by line after stripping ANSI escapes. Only lines beginning with the report prefix are parsed; everything else is passed through as human-readable log. The two record types are matched by order ID, and a report record without its meta record is rejected rather than guessed at.

Validation is where the bridge earns its place. `validateReport` checks that the order ID matches the 32-byte hex pattern, that the outcome is one of `DELIVERED` or `LOST`, that the payload is well-formed hex, that it decodes as the four-field receiver ABI to the same order and outcome, that the allowlist either is empty or contains the order, and that the report ID is not already in the state file. Anything that fails is skipped with a reason and never reaches calldata. The allowlist is the mechanism that made single-order demonstrations possible: with `BRIDGE_ORDER_ALLOWLIST` set to one ID, other shipments that CRE legitimately reported were logged as `NOT IN ALLOWLIST` and never built into a transaction.

Submission reads the escrow first — status, buyer, seller and amount — so that the log can state what will change and so that an unfunded order is refused before any gas is spent. The transaction is sent to `processReport(metadata, report)` with the exact metadata and payload CRE produced, and the receipt is awaited with a bounded timeout. On success the bridge records the report ID, order ID, transaction hash, block number, outcome and timestamp in `.local-bridge-state.json`, which is git-ignored. On revert it decodes the revert data to a readable reason, distinguishes an already-processed report from a genuine failure, and does not retry: a reverted submission is reported and the process exits non-zero, because a blind retry after an ambiguous failure is exactly how duplicate settlements happen.

With `LOCAL_BRIDGE_DRY_RUN=true` — the default, and the value used for every exploratory run — the same parse, validation and preflight execute, the same log lines are produced, and the prepared transaction is printed with its tar-

get, calldata length, report ID and intended effect, but nothing is broadcast. The switch to `false` is per-invocation on the command line, never persisted. This two-key arrangement (CRE dry-run, bridge dry-run) meant that the entire pipeline could be verified end to end, including the exact calldata, before a single token moved.

10.9 Automatic courier demonstration

`courier-scheduler.ts` is an optional process that makes the demonstration self-driving. It polls the shipment API for records that are not in a terminal state, waits the configured interval for the current status, then applies the next status through the same authenticated `PATCH` route a human would use. The default timeline is ten seconds from `CREATED` to `SHIPPED`, ten from `SHIPPED` to `IN_TRANSIT`, and twenty from `IN_TRANSIT` to the final state, with each delay overridable by environment variable. The final state is derived from the tracking ID: a tracking ID ending in `-LOST` yields `LOST`, anything else yields `DELIVERED`. That convention is documented in the file header precisely so that it is not mistaken for a settlement rule — it is a courier fact generator, and the workflow still decides what a fact means.

The scheduler can also invoke the bridge after a status change, which closes the loop from "parcel moves" to "money moves" without a human. It serialises those invocations so that two shipments reaching a terminal state at the same moment cannot start two overlapping CRE simulations, and it passes the order allowlist through so that a demonstration remains scoped to the orders under test. In the runs reported here the bridge was invoked explicitly rather than by the scheduler, so that each settlement could be verified individually; the scheduler's automatic progression was exercised and observed in its own log, and its automatic invocation path is implemented and configurable rather than being claimed as demonstrated.

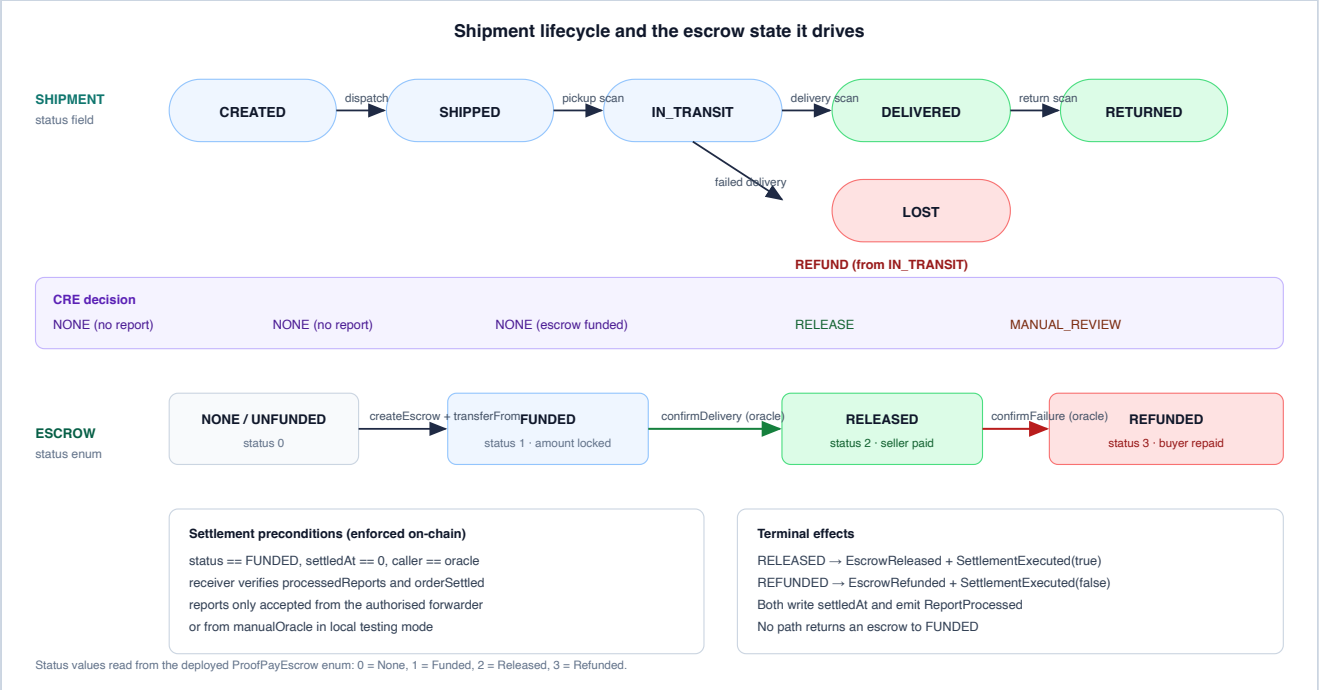


Figure 4: Shipment Lifecycle and the Escrow State It Drives

10.10 Smart contract design summary

Table 5 lists every element of the three contracts by kind. Names are reproduced exactly as they appear in the Solidity sources, so that the table can be used as a map when reading the code.

Table 5: Smart Contract Design

Component	Type	Purpose
ProofPayUSD	Contract	ERC-20 payment token ("ProofPay USD", pUSD, 18 decimals, zero initial supply) with an owner-only mint.

Component	Type	Purpose
`mint(address,uint256)`	Function	Creates pUSD for the owner; rejects the zero address through OpenZeppelin's inherited check.
ProofPayEscrow	Contract	Custody and settlement engine; inherits `Ownable` and `ReentrancyGuard`, uses `SafeERC20`.
`EscrowStatus`	Enum	`None`, `Funded`, `Released`, `Refunded` — the four states a deposit can occupy.
`Escrow`	Struct	`orderId`, `buyer`, `seller`, `amount`, `createdAt`, `settledAt`, `status`.
`paymentToken`, `oracle`	State variables	Immutable token reference; mutable settlement authority, set by the owner.
`_escrows`	Mapping	`bytes32 => Escrow`, private; reads go through `getEscrow`.
`createEscrow`	Function	Validates parties and amount, writes the struct as `Funded`, pulls tokens with `safeTransferFrom`.
`confirmDelivery` / `confirmFailure`	Functions	Pay the seller or refund the buyer; both `onlyOracle` and `nonReentrant`, both require `Funded`.
`getEscrow`	Function	Returns the struct; reverts with `EscrowNotFound` rather than returning a zeroed record.
`setOracle`	Function	Owner-only rotation of the settlement authority, with an event.
`onlyOracle`, `nonReentrant`	Modifiers	Restrict settlement to the receiver; guard every state-changing function against reentrancy.
`EscrowCreated`, `EscrowReleased`, `EscrowRefunded`, `OracleUpdated`	Events	Order ID indexed in all four; the settlement events also index the payee.
Ten custom errors	Errors	`InvalidToken`, `InvalidOrderId`, `InvalidBuyer`, `InvalidSeller`, `SameBuyerAndSeller`, `InvalidAmount`, `InvalidOracle`, `EscrowAlreadyExists`, `EscrowNotFound`, `EscrowNotFunded`, `UnauthorizedOracle` (eleven including the authorisation error).
ProofPayCREReceiver	Contract	Authorisation, decoding, replay protection and single-settlement enforcement in front of the escrow.
`processedReports`, `orderSettled`	Mappings	`bytes32 => bool`; block a repeated report and a second settlement of one order.
`onReport` / `processReport`	Functions	CRE forwarder entry point and local relay entry point; both `onlyAuthorized`, both delegate to `_processReport`.
`setForwarder`, `setManualOracle`, `supportsInterface`	Functions	Owner-only rotation of the two authorised callers; ERC-165 advertisement of `IReceiver`.
`ReportProcessed`, `SettlementExecuted`, `ForwarderUpdated`, `ManualOracleUpdated`	Events	Audit trail: outcome string and report ID on the settlement events; release flag on `SettlementExecuted`.

Component	Type	Purpose
Receiver errors	Errors	`UnauthorizedCaller`, `InvalidForwarder`, `InvalidOracle`, `ReportAlreadyProcessed`, `OrderAlreadySettled`, `EscrowNotFunded`, `EscrowNotFound`, `UnsupportedOutcome`, `InvalidReportData`.
IEscrowSettlement / IReceiver	Interfaces	Minimal external contracts: two settlement calls plus `getEscrow`, and the `onReport(bytes,bytes)` signature a DON expects.

10.11 Key code snippets

The excerpts below are taken verbatim from the sources, shortened to the lines that carry the logic. They are included so that the claims made in this report can be checked against the code without opening the repository.

contracts/contracts/ProofPayEscrow.sol — funding (lines 79–118)

```
function createEscrow(bytes32 orderId, address buyer, address seller, uint256 amount)
    external nonReentrant
{
    if (orderId == bytes32(0)) revert InvalidOrderId();
    if (buyer == address(0) || buyer != msg.sender) revert InvalidBuyer();
    if (seller == address(0)) revert InvalidSeller();
    if (buyer == seller) revert SameBuyerAndSeller();
    if (amount == 0) revert InvalidAmount();
    if (_escrows[orderId].status != EscrowStatus.None) revert EscrowAlreadyExists(orderId);

    _escrows[orderId] = Escrow({
        orderId: orderId, buyer: buyer, seller: seller, amount: amount,
        createdAt: uint64(block.timestamp), settledAt: 0, status: EscrowStatus.Funded
    });

    paymentToken.safeTransferFrom(buyer, address(this), amount);
    emit EscrowCreated(orderId, buyer, seller, amount);
}
```

contracts/contracts/ProofPayEscrow.sol — release (lines 119–128)

```
function confirmDelivery(bytes32 orderId) external onlyOracle nonReentrant {
    Escrow storage escrow = _getFundedEscrow(orderId);
    escrow.status = EscrowStatus.Released;
    escrow.settledAt = uint64(block.timestamp);
    paymentToken.safeTransfer(escrow.seller, escrow.amount);
    emit EscrowReleased(orderId, escrow.seller, escrow.amount);
}
```

contracts/contracts/ProofPayCREReceiver.sol — authorisation (lines 80–96)

```
modifier onlyAuthorized() {
    if (msg.sender == forwarder) { _; return; }
    if (manualOracle != address(0) && msg.sender == manualOracle) { _; return; }
    if (owner() == msg.sender && forwarder == address(0)) { _; return; }
    revert UnauthorizedCaller(msg.sender);
}
```

contracts/contracts/ProofPayCREReceiver.sol — replay and duplicate protection (lines 138–152)

```
bytes32 reportId = keccak256(
    abi.encodePacked(orderId, outcome, trackingIdHash, observedAt, metadata)
);
if (processedReports[reportId]) revert ReportAlreadyProcessed(reportId);
if (orderSettled[orderId]) revert OrderAlreadySettled(orderId);

try this._checkEscrowFunded(orderId) returns (bool funded) {
    if (!funded) revert EscrowNotFunded(orderId);
} catch { revert EscrowNotFound(orderId); }
```

chainlink/cre/shipment-workflow/main.ts — decision table (lines 77–110)

```
function shouldSettle(shipment: { status: string }): SettlementDecision {
  switch (shipment.status) {
    case "DELIVERED":
      return { action: "RELEASE", reason: "Shipment delivered - eligible for seller payment", outcome:
"DELIVERED" };
    case "LOST":
      return { action: "REFUND", reason: "Shipment lost - eligible for buyer refund", outcome: "LOST"
};
    case "IN_TRANSIT":
    case "SHIPPED":
      return { action: "NONE", reason: "Shipment in transit/shipped - payment remains locked" };
    case "RETURNED":
      return { action: "MANUAL_REVIEW", reason: "Returned after delivery - requires manual review" };
    case "CREATED":
      return { action: "NONE", reason: "Shipment just registered - not ready for settlement" };
    default:
      return { action: "NONE", reason: `Unknown shipment status: ${shipment.status}` };
  }
}
```

chainlink/cre/shipment-workflow/main.ts — report generation precedes the dry-run branch (lines 341–345, 392–395)

```
const reportRequest = prepareReportRequest(reportPayload);
const report = runtime.report(reportRequest).result();
const { payloadHex, payloadSource, creRawReportHex } =
    extractAbiPayload(report, orderIdBytes32, decision.outcome);
// ... structured PROOFPAY_REPORT_V1 emission happens here ...
if (config.dryRun) {
    runtime.log(`DRY RUN: Report generated for order ${shipment.orderId}; CRE write skipped`);
    continue;
}
```

chainlink/cre/shipment-workflow/local-runner.ts — bridge configuration defaults (selected lines, 211–218)

```
rpcUrl: process.env.HARDHAT_RPC_URL ?? process.env.LOCAL_RPC_URL ?? "http://127.0.0.1:8545",
expectedChainId: Number(process.env.BRIDGE_EXPECTED_CHAIN_ID ?? 31337),
allowedOrderIds: parseOrderAllowlist(process.env.BRIDGE_ORDER_ALLOWLIST),
// ...
dryRun: envFlag("LOCAL_BRIDGE_DRY_RUN", true),
```

chainlink/cre/shipment-workflow/courier-scheduler.ts — courier fact generation (lines 158–163)

```
case "CREATED":      return "SHIPPED";
case "SHIPPED":      return "IN_TRANSIT";
case "IN_TRANSIT":   return trackingId.toUpperCase().endsWith("-LOST") ? "LOST" : "DELIVERED";
```

10.12 Deployment details

Table 6 records the deployed addresses, the transactions that created them and the wiring read back from the contracts. Deployment gas is the actual gas consumed, read from each creation receipt rather than estimated. Transaction hashes are reproduced in full in Appendix A; addresses are given in full here because they are needed to reproduce the checks.

Table 6: Deployment Details

Item	Details
Network	Hardhat Local (in-process node exposed as `localhost`)
Chain ID	31337
RPC endpoint	`http://127.0.0.1:8545`
pUSD token address	`0x5FbDB2315678afecb367f032d93F642f64180aa3`

Item	Details
pUSD deployment	Block 1, gas 618,390, tx `0xb10fd1938748aa3d1cd691525cccd3a6857772b93dbc3e9fa44e3179cd1c0116`
Escrow address	`0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512`
Escrow deployment	Block 2, gas 815,030, tx `0x126d912777073a40828533d031b5c25274b0b33650a8b6d5f8cd38295b573fb1`
Receiver address	`0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e`
Receiver deployment	Block 18, gas 880,716, tx `0x2c1b12ddabf5a951c26eaef23f2bbaa8c3db0df6210affa9506bf46b0991ee7d`
`escrow.oracle()`	`0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e` (equals the receiver), set in block 19 (gas 30,539)
`receiver.manualOracle()`	`0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266` (Hardhat account 0, the deployer and owner)
`receiver.forwarder()`	`0x0000000000000000000000000000000000000000` — no DON forwarder configured, so the manual path is active
Owner	`0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266` for all three contracts
Compiler	Solc 0.8.28, optimizer enabled, 200 runs
Deployment record file	`contracts/deployments/cre-receiver-localhost.json` (network, chain ID, deployer, owner, escrow, forwarder, manual oracle, receiver, timestamp)
Source repository	Not available in the current implementation — no public GitHub URL is configured in the project metadata.

11. RESULTS

This section reports outcomes that were measured, not projected. Test counts come from runner output, settlement figures come from transaction receipts read back over RPC, and gas figures are the gas actually consumed. Where a value could not be obtained it is marked as unavailable rather than estimated.

11.1 Smart contract test results

All three contract suites pass. The runner reports 31 passing tests in 727 ms with zero failures: 9 for `ProofPayCREReceiver`, 16 for `ProofPayEscrow` and 6 for `ProofPayUSD`. The receiver suite covers authorisation, report decoding, replay rejection, duplicate settlement rejection, unknown and unfunded orders, and independent settlement of multiple escrows. The escrow suite covers funding, both payout paths, custom-error arguments for every validation branch, oracle rotation, and a reentrancy case that attempts to re-enter `createEscrow` from a malicious token. The token suite covers metadata, supply, owner-only minting, standard approval and transfer semantics, insufficient balance, and zero-address construction.

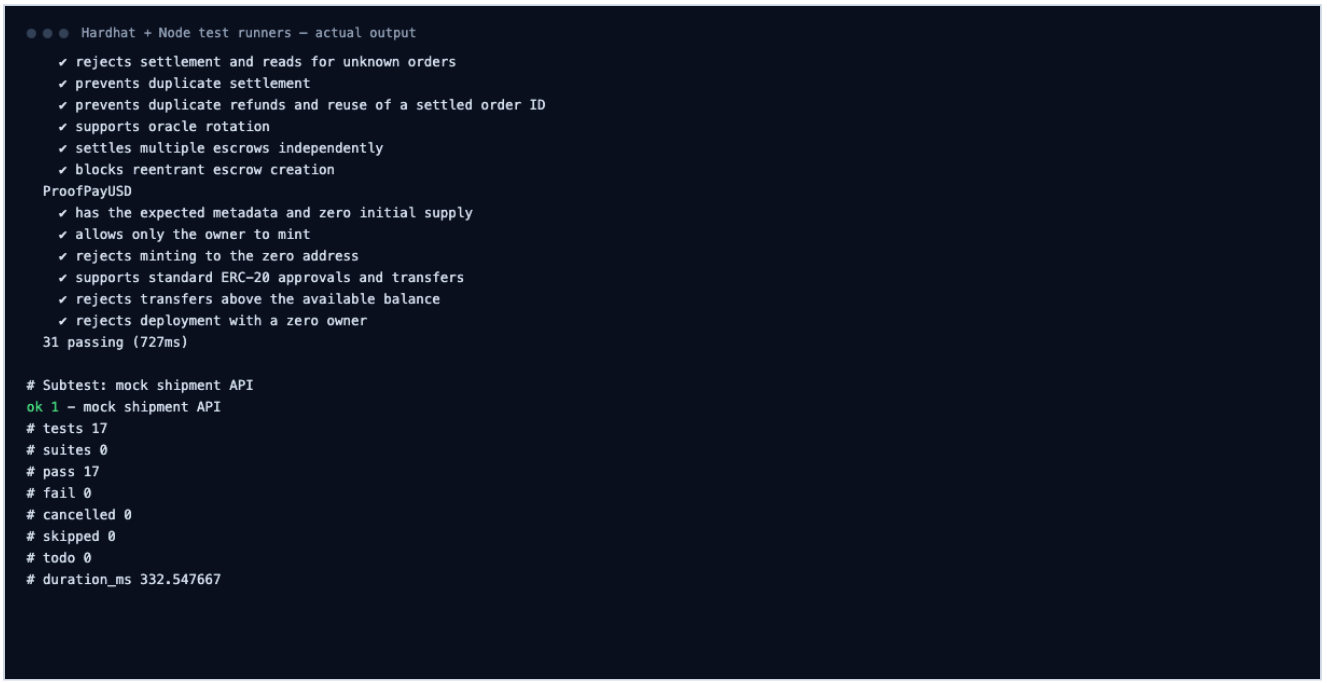


Figure 5: Contract Test Suite Output (31 Passing) and Backend Test Summary (17 Passing)

The visible tail in Figure 5 shows the negative cases that matter most for this design: "prevents duplicate settlement", "prevents duplicate refunds and reuse of a settled order ID", "supports oracle rotation", "settles multiple escrows independently" and "blocks reentrant escrow creation". Below the Hardhat output, the Node test runner reports 17 passing tests in 332 ms for the backend.

11.2 Backend test results

The shipment API suite uses Node's built-in test runner against an in-memory instance of the service. All 17 tests pass with zero failures and zero skipped. The cases cover registration validation (malformed order ID, missing tracking ID, duplicate order), the full forward transition path, rejection of an illegal transition, rejection of a change to a terminal record, event-history append correctness, single-record lookup for an unknown ID, and the funding precondition that refuses a shipment whose escrow is not funded. The suite runs in about a third of a second, so it is cheap enough to run on every change.

11.3 Frontend results

TypeScript compilation completes with no diagnostics under the project's strict configuration, and the Vite production build completes successfully, emitting a bundle to `frontend/dist`. Both commands were re-run for this report rather than recalled from earlier work. In the browser, wallet connection, network detection, balance display, escrow discovery from logs, per-order reads, the approve-and-fund sequence and the shipment timeline were all exercised against the running local node. Two limitations of this verification should be stated: MetaMask itself cannot be automated headlessly, so the dashboard screenshots in Section 11.7 were captured through a read-only EIP-1193 shim that forwards `eth_call`, `eth_getLogs` and `eth_chainId` to the same node and returns no accounts; and no click-through interaction test suite exists in the repository, so the UI claims rest on manual exercise plus a clean type check and build.

11.4 Actual blockchain settlement results

Five calls to `processReport` exist on the local chain, at blocks 20, 21, 24, 29 and 30. Blocks 20 and 21 were the first receiver-mediated settlements used to validate the mechanism; blocks 24, 29 and 30 are the three documented demonstration settlements, each produced by a CRE-generated report relayed by the bridge. In every case the receipt status is 1, the escrow moved from `Funded` to its terminal state, and the expected party's pUSD balance changed by exactly the escrowed amount. Table 9 lists the three demonstration settlements with their full transaction hashes; Appendix A adds the deployment and control transactions.

Table 9: Executed CRE-Mediated Settlements

Test	Outcome	Order ID	Amount	Block / gas	Transaction hash
TC-1	DELIVERED → RELEASE	`0x4d073cded438abe78792ce1e8dd508f73a1d5d5ceb05bf2e93adf70575e3aaf3`	100 pUSD	24 / 122,471	`0x78af4106dfa323c13640f75a0e0fd8ec54320742cc5c6ba577adc45fc712bd6c`
TC-2	LOST → REFUND	`0x0c34a992899f022937db08abbb10b4b02c4a1dfd967b03aa74c4ef379e031152`	100 pUSD	29 / 122,425	`0x835890473e5d61dc4a1c8a27fdefd94f0777df26169554f1420534f0670a5288`
TC-3	DELIVERED → RELEASE	`0xf694b593b682ac612e6f5c784a6801deab4f4c0b7420272b555d8ce669c26434`	100 pUSD	30 / 122,471	`0x74d678211c74efda145b9c712adae35bcd5a35616275ec8d76d0dcc3a232f4a6`

TC-1 — release after delivery. Order `0x4d07…aaf3` was funded with 100 pUSD and its tracking ID `PHASE-A-DELIVERED-1` was advanced to `DELIVERED`. The simulation logged "Generating DELIVERED report", emitted the structured record with a 256-byte payload, and printed "DRY RUN: Report generated ... CRE write skipped" — confirming that the workflow produced a real report while its own on-chain write stayed disabled. The bridge parsed that payload, verified it decoded to the order and outcome, submitted `processReport`, and recorded receipt status 1 at block 24 with 122,471 gas. The escrow moved to `Released` with `settledAt` 1793560884, and the seller's pUSD balance rose by exactly 100.

TC-2 — refund after loss. Order `0x0c34…1152` with tracking ID `PPD-DEMO-LOST` was reported `LOST`. The CRE simulation returned "Processed 2 settlements", emitting two report records: this order as `LOST / ELIGIBLE` and the delivered order as `DELIVERED / NOT IN ALLOWLIST`. Because the allowlist contained only the lost order, exactly one transaction was built. It confirmed at block 29 with 122,425 gas, the escrow became `Refunded` with `settledAt` 1793561715, and the buyer's balance rose by exactly 100 while the seller's was unchanged. This run is the clearest evidence that the allowlist and the skip logic work: CRE legitimately reported two shipments and only one was ever submitted.

TC-3 — release after delivery, second order. Order `0xf694…c26434` with tracking ID `PPD-DEMO-DELIVERED` was reported `DELIVERED`. This time CRE emitted a single report — the lost order was already refunded, so the workflow skipped it on the escrow status check — and the bridge submitted it at block 30 with 122,471 gas. The escrow became `Released`, the seller's balance rose by 100 to 5,993 pUSD, and the buyer's was unchanged.

Two negative results are worth recording. First, the order `0x4d07…aaf3` was reported once and settled once; the bridge's state file holds three entries, one per successful submission, and no report ID appears twice. Second, none of the three runs advanced the block height by more than one, which rules out a hidden extra transaction such as an approval or a retry. After the final settlement the RPC reports block 30 for the demonstration runs; a subsequent read at block 32 reflects a later, unrelated escrow created through the UI, which is why the balance snapshot quoted below differs from the balance visible in the settlement log.

```
on-chain settlement verification — actual output
ProofPay local settlement verification
RPC http://127.0.0.1:8545   chainId 31337   block 32

TC-1  DELIVERED -> RELEASE   order 0x4d073cded438abe7...e3aaf3
  tx        0x78af4106dfa323c13640f75a0e0fd8ec54320742cc5c6ba577adc45fc712bd6c
  receipt   status=1  block=24  gasUsed=122471
  escrow    status=2  settledAt=1793560884  amount=100.0 pUSD
  receiver  orderSettled=true  reportId=0x12ccb1955fc932c7a4e695a5643b1167df1b307ac65515a805a45db06e9c751c

TC-2  LOST -> REFUND   order 0x0c34a992899f0229...031152
  tx        0x835890473e5d61dc4a1c8a27fdefd94f0777df26169554f1420534f0670a5288
  receipt   status=1  block=29  gasUsed=122425
  escrow    status=3  settledAt=1793561715  amount=100.0 pUSD
  receiver  orderSettled=true  reportId=0x774fb352cbdd3c0100ef80b53cc77ccc68f72cdb346654fd885e3661bf225ea4

TC-3  DELIVERED -> RELEASE   order 0xf694b593b682ac61...c26434
  tx        0x74d678211c74efda145b9c712adae35bcd5a35616275ec8d76d0dcc3a232f4a6
  receipt   status=1  block=30  gasUsed=122471
  escrow    status=2  settledAt=1793561791  amount=100.0 pUSD
  receiver  orderSettled=true  reportId=0x320fd2a99bd17b625a11298af2a1b1b71701990e6f9cefe259631f4eb4c3c3bf

balances  buyer=608.0 pUSD   seller=5993.0 pUSD
state entries 3   (no duplicate submissions)
```

Figure 6: On-Chain Verification of the Three Settlements Read Back from Hardhat

Figure 6 is the verification script's output rather than the bridge's own log, so it is an independent read: it queries `getEscrow`, `orderSettled` and `balanceOf` after the fact and prints what the chain says. The three report IDs it shows are the values stored in `processedReports`, and the three transaction hashes match Table 9.

```
cre workflow simulate — actual stdout
2026-11-02T00:41:26Z [SIMULATION] Running trigger trigger=cron-trigger@1.0.0
2026-11-02T00:41:26Z [USER LOG] ProofPay CRE: Checking shipments from http://localhost:5001
2026-11-02T00:41:26Z [USER LOG] Shipment PHASE-A-DELIVERED-1: Generating DELIVERED report
2026-11-02T00:41:26Z [USER LOG] PROOFPAY_REPORT_V1:
{"type":"report","orderId":"0x4d073cded438abe78792ce1e8dd508f73a1d5d5ceb05bf2e93adf70575e3aaf3","outcome":"DELIVERED","trackingId":"PHASE-A-DELIVERED-
1","observedAt":"2026-11-
01T19:09:39.775Z","metadata":"0x","report":"0x4d073cded438abe78792ce1e8dd508f73a1d5d5ceb05bf2e93adf70575e3aaf3000000000000000000000000000000000000000000000000
0000000000000080d8c77f28d8965071e4daccbe3545227c02bfe85fdd3e012e287a8b915e783b0d0000000000000000000000000000000000000000000000000000000000000c00000000000
0000000000000000000000000000000000000000000000000000000000944454c4956455245440000000000000000000000000000000000000000000000000000000000000000000000000
0000000000000000000000018323032362d31312d30315431393a30393a33392e3737355a000000000000000"}
2026-11-02T00:41:26Z [USER LOG] PROOFPAY_REPORT_V1:
{"type":"meta","orderId":"0x4d073cded438abe78792ce1e8dd508f73a1d5d5ceb05bf2e93adf70575e3aaf3","action":"RELEASE","trackingIdHash":"0xd8c77f28d8965071e4daccb
e3545227c02bfe85fdd3e012e287a8b915e783b0d","reportSource":"body","reportDecodesToReceiverAbi":true,"creRawReportLength":365,"creMetadataHeaderLength":109,"c
reRawReportHash":"0xbec77a1be3e8ea34252202280ba47b450f9dbd2c3e8ae55c9ba39d776d483bb6","receiver":"0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e","escrow":"0xe7
f1725E7734CE288F8367e1Bb143E90bb3F0512","creDryRun":true}
2026-11-02T00:41:26Z [USER LOG] DRY RUN: Report generated for order 0x4d073cded438abe78792ce1e8dd508f73a1d5d5ceb05bf2e93adf70575e3aaf3; CRE write skipped
2026-11-02T00:41:26Z [USER LOG] Shipment PP-LOST-002: Escrow not funded (status 3), skipping
2026-11-02T00:41:26Z [USER LOG] Shipment PP-SUCCESS-002: Escrow not funded (status 2), skipping
2026-11-02T00:41:26Z [USER LOG] Shipment PP-SUCCESS-001: Escrow not funded (status 2), skipping
2026-11-02T00:41:26Z [USER LOG] Error processing shipment UI-MVWO84Z8: [2]Unknown: Error: VM Exception while processing transaction: reverted with custom
error 'EscrowNotFound("0x9df3f74c30e80532c815a06efbc8ff3f445d10d198ef017703b8b1a9c01d62d7")'
2026-11-02T00:41:26Z [USER LOG] Shipment UI-MVWNKIAB: Shipment in transit/shipped — payment remains locked
2026-11-02T00:41:26Z [USER LOG] Shipment P3-CONCURRENT-MVWNDRN1: Shipment in transit/shipped — payment remains locked
2026-11-02T00:41:26Z [USER LOG] Shipment P3-RETURNED-MVWNDRN1: Returned after delivery — requires manual review
2026-11-02T00:41:26Z [USER LOG] Error processing shipment P3-LOST-MVWNDRN1: [2]Unknown: Error: VM Exception while processing transaction: reverted with
custom error 'EscrowNotFound("0xd6d808063dcafc26a5c0c115b303e335224bba5bcf579d53971cfd71bd8a4362")'
2026-11-02T00:41:26Z [USER LOG] Error processing shipment P3-DELIVERED-MVWNDRN1: [2]Unknown: Error: VM Exception while processing transaction: reverted with
custom error 'EscrowNotFound("0xb294f31699ea29815fdec43fbf9a16d8789bae9bdab6d75e97a1e9bbf6755dd9")'
```

Figure 7: CRE Workflow Simulation Output Showing the Structured Report Records and Dry-Run Skip

Figure 7 captures the workflow's own stdout. The first record is the report payload — order ID, outcome `DELIVERED`, tracking ID, observation timestamp, empty metadata and the 256-byte ABI payload as hex. The second record is the provenance companion, with `reportSource: "body"`, `reportDecodesToReceiverAbi: true`, `creRawReportLength: 365`, `creMetadataHeaderLength: 109`, `creDryRun: true`, and the configured receiver and escrow addresses. Below them, the run shows the behaviours that keep the system safe: already-settled shipments are skipped ("Escrow not funded (status 2)"), in-transit shipments produce no report, a returned shipment is routed to manual review, and shipments whose escrow does not exist fail with `EscrowNotFound` and are logged rather than crashed. The final line prints "DRY RUN", which is the proof that no CRE write occurred.

```
local-runner.ts — actual dry-run stdout
ProofPay local CRE execution bridge
  Mode: CRE workflow simulation + local report relay (not DON execution)
  Bridge mode: DRY RUN (no transaction will be sent)
  RPC: http://127.0.0.1:8545
  Receiver: 0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e
  Escrow:   0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
  Signer:   0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (unlocked Hardhat account, no private key)
  Allowlist: 0x0c34a992899f022937db08abbb10b4b02c4a1dfd967b03aa74c4ef379e031152 (only these orders may be settled)
Preflight
  chainId            31337 ok
  receiver code      3583 bytes ok
  escrow code        3274 bytes ok
  admin unlocked     ok 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (rpc accounts: 20)
  receiver.oracle    0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 ok (== signer)
  escrow.oracle      0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e ok (== receiver)
  admin balance      9999.99628763362650112 ETH ok
  all checks passed
Running CRE workflow
$ cre workflow simulate shipment-workflow --target staging-settings --trigger-index 0 --non-interactive
  (cwd: /Users/chiranthml/Desktop/Minor_2/chainlink/cre)
  CRE result: "Processed 2 settlements"
CRE emitted 2 report(s):
  0x0c34a992899f022937db08abbb10b4b02c4a1dfd967b03aa74c4ef379e031152  LOST  ELIGIBLE
  0xf694b593b682ac612e6f5c784a6801deab4f4c0b7420272b555d8ce669c26434  DELIVERED  NOT IN ALLOWLIST — will not be submitted
validating eligible reports
Report 0x0c34a992…031152 (LOST)
  escrow seller      0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
  escrow amount      100 pUSD
  receiver           0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e
  metadata           0x (0 bytes)
  report             256 bytes
  calldata           388 bytes -> processReport(bytes,bytes)
  reportId           0x774fb352cbdd3c0100ef80b53cc77ccc68f72cdb346654fd885e3661bf225ea4
  CRE action         REFUND
```

Figure 8: Local Execution Bridge in Dry-Run Mode — Preflight Passed, Calldata Prepared, Nothing Broadcast

Figure 8 is the bridge's dry-run output for the lost order. It shows the mode line ("CRE workflow simulation + local report relay (not DON execution)"), the bridge mode "DRY RUN (no transaction will be sent)", the RPC and the three addresses, and the allowlist containing exactly one order. The preflight block reports chain ID 31337, receiver code 3,583 bytes, escrow code 3,274 bytes, the admin account unlocked with 20 RPC accounts, `receiver.manualOracle == signer`, `escrow.oracle == receiver`, and a native balance sufficient for gas. It then shows the simulation result "Processed 2 settlements", the two parsed reports with the second marked "NOT IN ALLOWLIST — will not be submitted", the escrow snapshot (seller, 100 pUSD), the 256-byte report, the 388-byte calldata for `processReport(bytes,bytes)`, the derived report ID and the intended CRE action `REFUND`. Nothing was broadcast in this run; the block height was checked before and after and was unchanged.

11.5 Test cases

Table 7 lists the cases that were actually exercised and the evidence for each. Cases that were not run are marked as such rather than assumed to pass, and the "Actual result" column quotes the observed outcome rather than restating the expectation.

Table 7: Test Cases

TC ID	Test scenario	Input	Expected result	Actual result	Status
TC-01	Token deployment metadata	Deploy with the owner account	Name "ProofPay USD", symbol pUSD, 18 decimals, supply 0	All four match; owner is the deployer	Pass
TC-02	Owner-only minting	Mint from a non-owner account	Revert `OwnableUnauthorizedAccount`	Reverted with that error and the caller's address	Pass
TC-03	Escrow funding	approve 100 pUSD, then createEscrow	Escrow `Funded`; tokens held by contract; event emitted	Status 1, balance moved, `EscrowCreated` emitted	Pass
TC-04	Duplicate funding of one order	createEscrow twice with the same order ID	Revert `EscrowAlreadyExists`	Reverted with the order ID	Pass
TC-05	Delivery release through the receiver	CRE report, outcome DELIVERED	Seller paid exactly the escrowed amount; status Released	Block 24 and block 30: seller +100 pUSD each	Pass
TC-06	Lost-shipment refund	CRE report, outcome LOST	Buyer repaid; status Refunded	Block 29: buyer +100 pUSD, seller unchanged	Pass
TC-07	Unauthorised settlement attempt	processReport from an unrelated account	Revert `UnauthorizedCaller`	Reverted in the receiver suite	Pass
TC-08	Duplicate report replay	Same report bytes submitted twice	Revert `ReportAlreadyProcessed`	Reverted with the same report ID	Pass
TC-09	Second settlement of one order	Different report for an already settled order	Revert `OrderAlreadySettled`	Reverted with the order ID	Pass
TC-10	Settlement of an unknown order	Report for an order with no escrow	Revert `EscrowNotFound`	Reverted; CRE also logged the same condition and skipped	Pass
TC-11	Shipment in transit	Status SHIPPED / IN_TRANSIT	No report, escrow remains funded	CRE logged "payment remains locked"	Pass

TC ID	Test scenario	Input	Expected result	Actual result	Status
TC-12	Returned shipment	Status RETURNED	No automatic settlement; manual review	CRE logged "requires manual review"	Pass
TC-13	CRE report generation	Eligible funded order	256-byte payload decoding to order and outcome	Payload 256 bytes, raw report 365 bytes, decodes correctly	Pass
TC-14	Bridge dry-run containment	LOCAL_BRIDGE_DRY_RUN=true	Calldata printed, no transaction sent	Block height unchanged; summary "planned", exit 0	Pass
TC-15	Order allowlist enforcement	Two CRE reports, allowlist of one order	Only the allowlisted order submitted	"skipped 1, submitted 1" in the lost-order run	Pass
TC-16	Frontend type check and build	`tsc` and `vite build`	No diagnostics; build succeeds	Both succeeded	Pass
TC-17	Backend validation and transitions	17 API tests	All pass, including illegal-transition rejections	17 passing, 0 failing	Pass
TC-18	Fully automatic courier-to-settlement loop	Scheduler invokes the bridge unattended	Settlement without human action	Scheduler progression verified; settlements were invoked explicitly, not by the scheduler	Not run

11.6 Gas analysis

Table 8 reports gas consumed, read from receipts. Deployment, allowance, minting and escrow creation were measured from the chain's early blocks; the settlement figures are the receiver transactions through which both payout paths are executed. The escrow's internal `safeTransfer` calls do not have separate receipts because they execute inside those transactions, so a figure for "ERC-20 transfer" alone is given only for the standalone transfers that exist on chain, and none is claimed for the internal ones.

Table 8: Gas Analysis

Function	Gas used	Remarks
Deploy ProofPayUSD	618,390	Block 1; includes ERC-20 and Ownable bytecode.
Deploy ProofPayEscrow	815,030	Block 2; largest of the three, includes struct, mapping and SafeERC20 paths.
Deploy ProofPayCREReceiver	880,716	Block 18; includes two mappings, interfaces and the escrow self-call check.
`setOracle`	47,639 / 30,539	Blocks 3 and 19; the first writes a previously empty slot, the second overwrites it.
`mint`	70,708 / 53,608 / 36,520	Blocks 4, 5 and 6; decreasing because the recipient's balance slot warms up.
`approve`	46,383	Nine identical approvals across blocks 7–31; standard ERC-20 allowance write.
`createEscrow`	173,232 first, 156,144 thereafter	Nine escrows. The first pays for cold storage of the mapping slot; the rest reuse it.
`processReport` (RELEASE)	122,471	Blocks 24 and 30; includes escrow check, payout transfer and four events.
`processReport` (REFUND)	122,425	Block 29; marginally cheaper than a release because the payee slot was already warm.
`confirmDelivery` (direct oracle call)	71,186	Block 11; the pre-CRE manual path, useful as a baseline for the receiver overhead.

Function	Gas used	Remarks
ERC-20 `transfer` (standalone)	Not available	No standalone transfer was sent in the recorded history; token movement occurs inside funding and settlement.

The comparison is informative. Routing a settlement through the receiver costs roughly 51,000 gas more than calling the escrow directly (122,471 against 71,186), which is the price of report decoding, the funding self-call, two mapping writes and two extra events. That overhead is fixed and small relative to the value it protects, and it buys the properties that make unattended settlement safe. On a layer-2 network the absolute cost would fall by one to two orders of magnitude, which is the main argument for the deployment path in Section 14.

11.7 Screenshots and visual results

The screenshots below were captured from the running application against the local node. Because MetaMask cannot be driven headlessly, the dashboards were rendered through a read-only provider shim that answers `eth_call`, `eth_getLogs` and `eth_chainId` from the same Hardhat node; the data on screen is therefore real chain state, while the wallet connection itself is stubbed. No interface element was mocked or invented and no pixel was altered; the tall captures are cropped from the top of the page only, so that interface text stays legible at print size, and each caption names what the visible region shows. The header balance in Figures 9–15 was captured at block 30, before the later escrow mentioned in Section 11.4; the block-32 snapshot is quoted separately in the text.

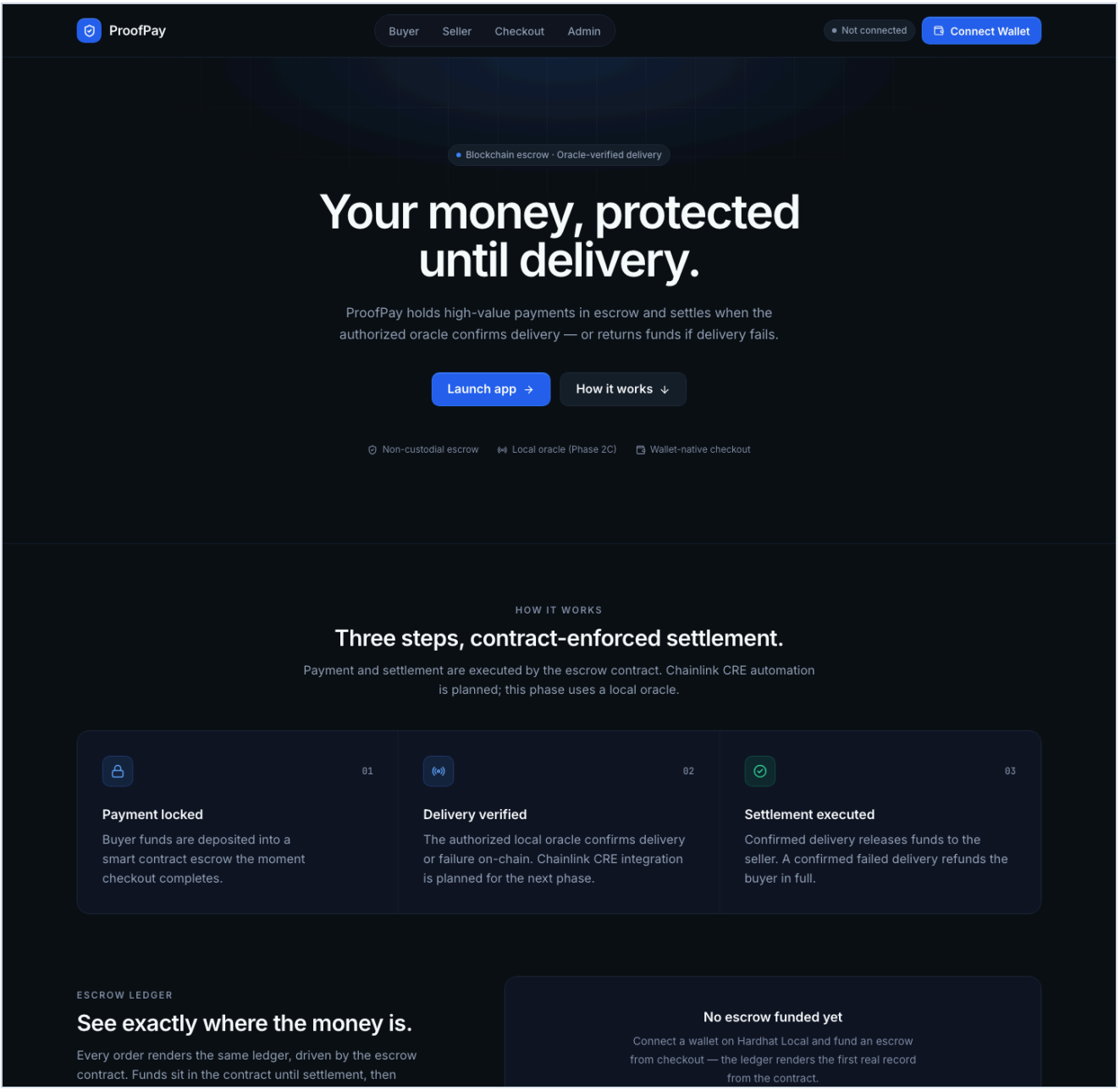


Figure 9: ProofPay Landing Page

The landing page introduces the product in financial terms — escrow protection for high-value commerce, funds settled only when delivery is verified — and states the environment honestly with a testnet banner naming Hardhat Local and warning that shipment tracking is simulated.

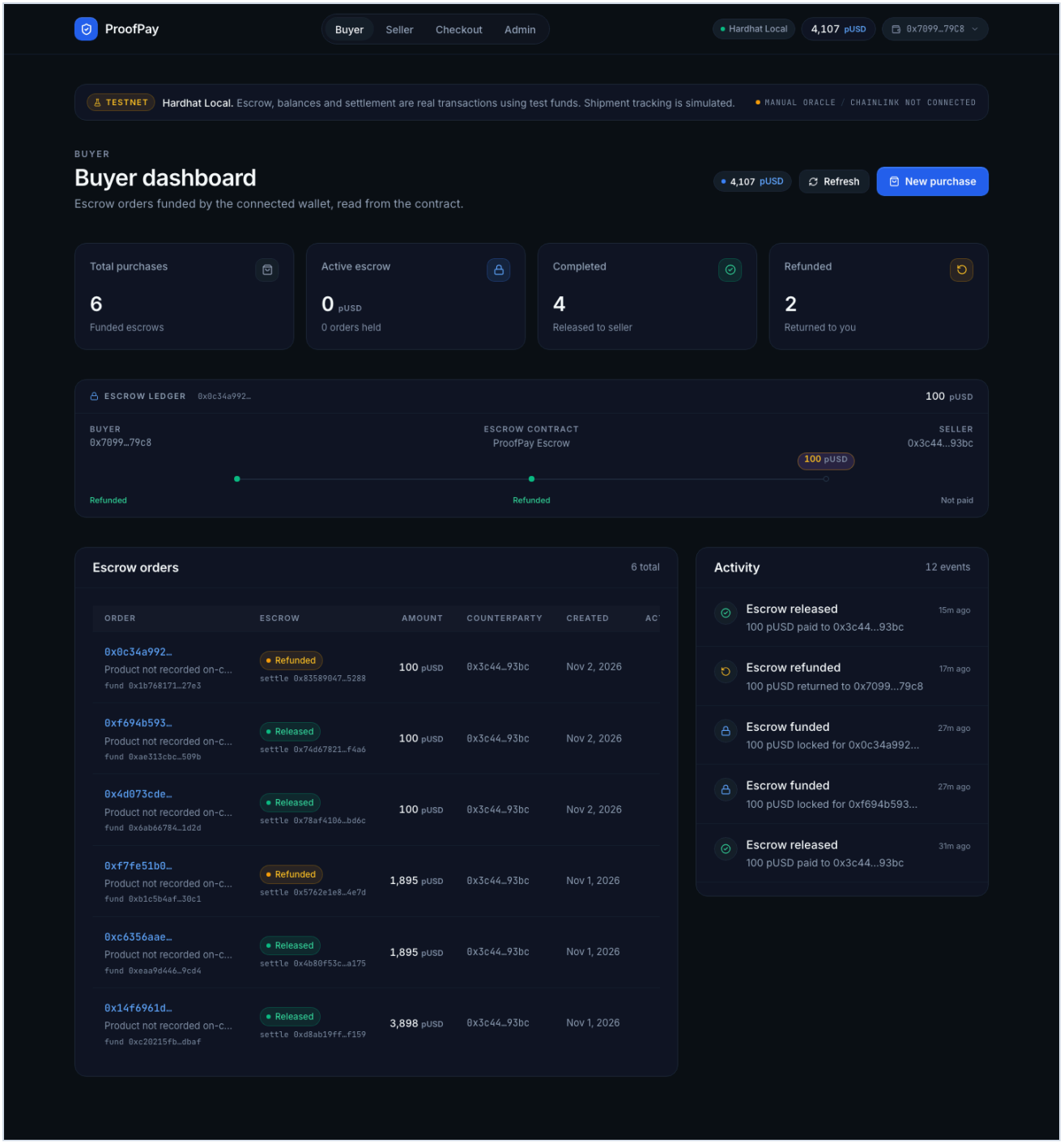


Figure 10: Buyer Dashboard with Live pUSD Balance and Escrow Activity

The buyer dashboard shows the connected account's pUSD balance, its escrow positions and an activity feed derived from escrow timestamps. Every figure is read from the chain at load time: the balance from `balanceOf`, the positions from `getEscrow` after discovery through `EscrowCreated` logs.

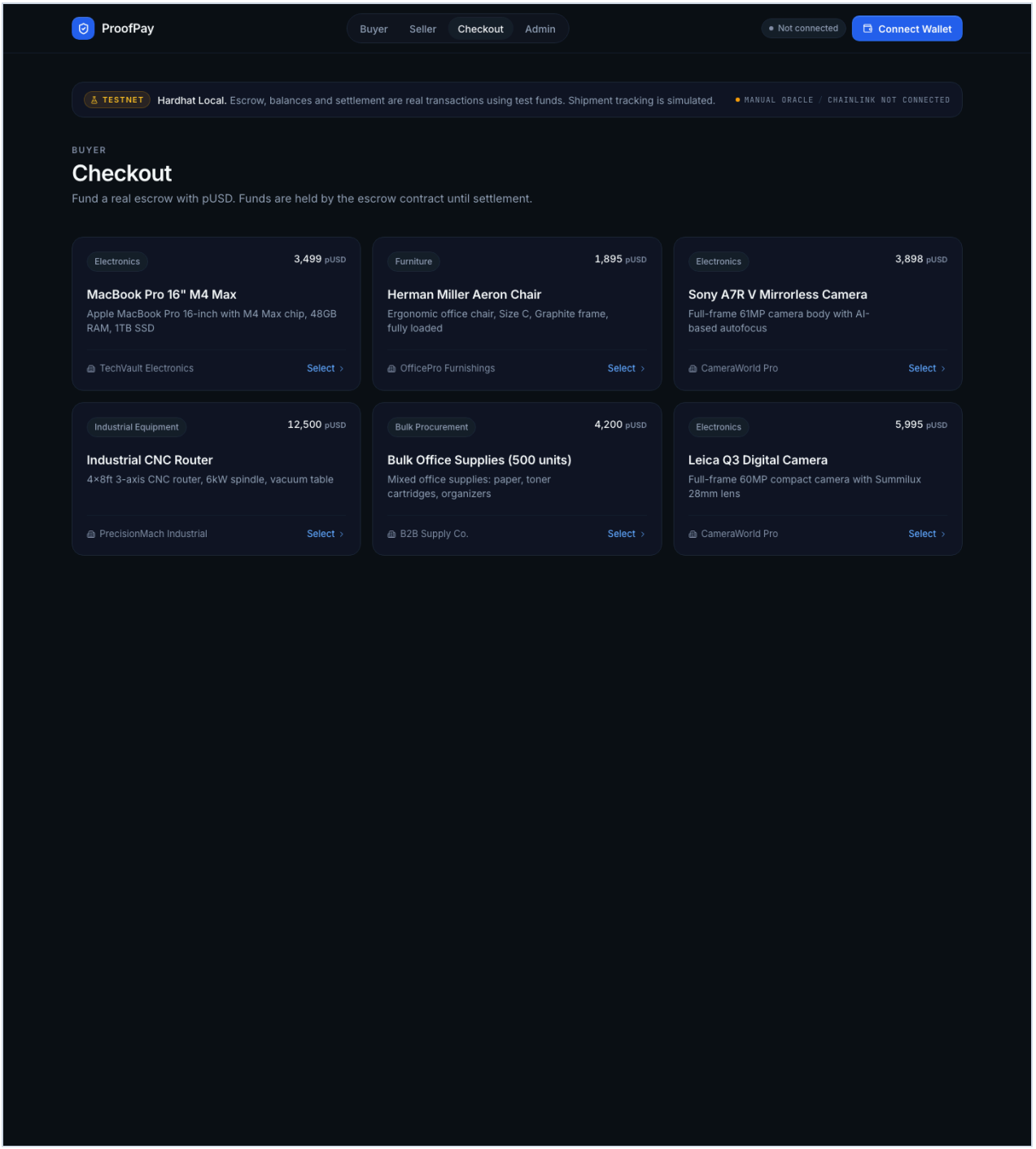


Figure 11: Checkout — Approve and Fund an Escrow

Checkout is the only place where a buyer creates value. The page shows the order summary, the allowance state and the two-step funding flow, and it refuses to offer the funding action when the wallet is on the wrong chain or holds insufficient pUSD.

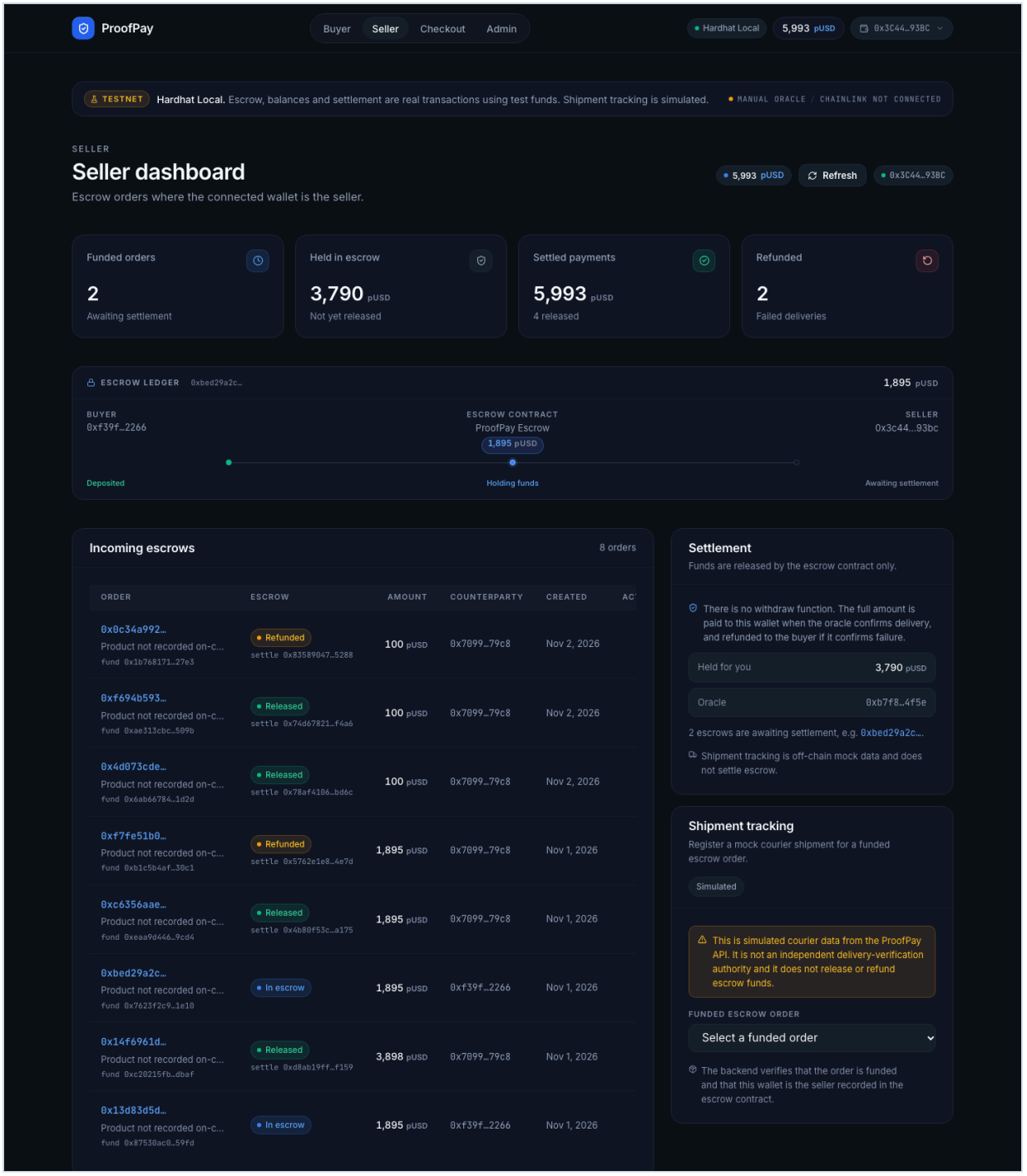


Figure 12: Seller Dashboard with Incoming Escrows and Settlement Status

The seller view aggregates the incoming escrows and shows, per order, whether the funds are held, released or refunded, together with the funding and settlement transaction hashes. The right-hand panel states the custody rule plainly: there is no withdrawal function, and the full amount is paid when the oracle confirms delivery or refunded if it confirms failure.

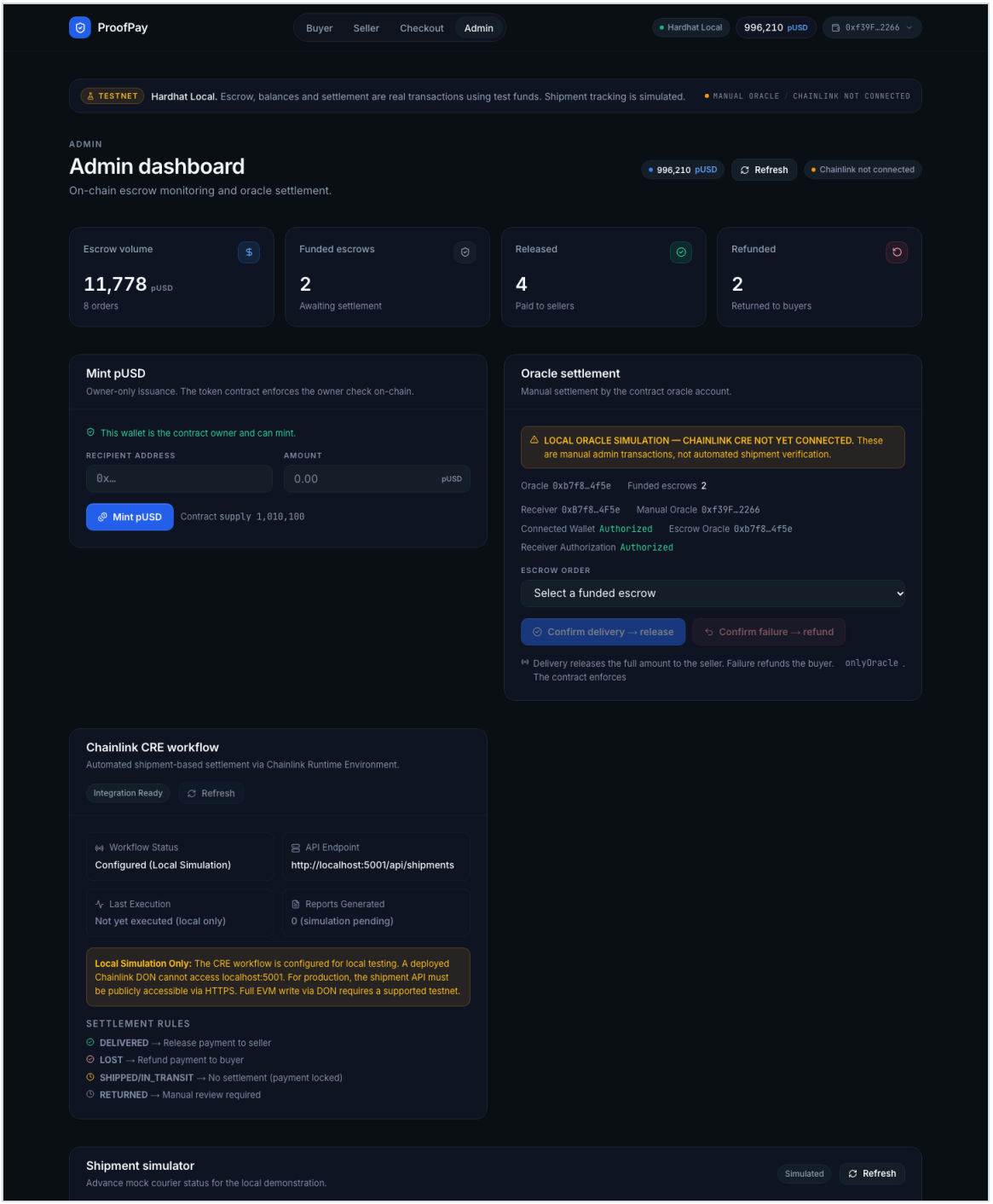


Figure 13: Admin Dashboard — Oracle Wiring, Shipment Simulator and Receiver Panel

The admin dashboard exposes the operational surface: the live `escrow.oracle()` and `receiver.manualOracle()` values, the shipment simulator for manual status changes, and the manual settlement panel that writes through the receiver. This is the path that existed before the CRE pipeline and it remains functional, which is what allows the automated path to be compared against a known-good manual one.

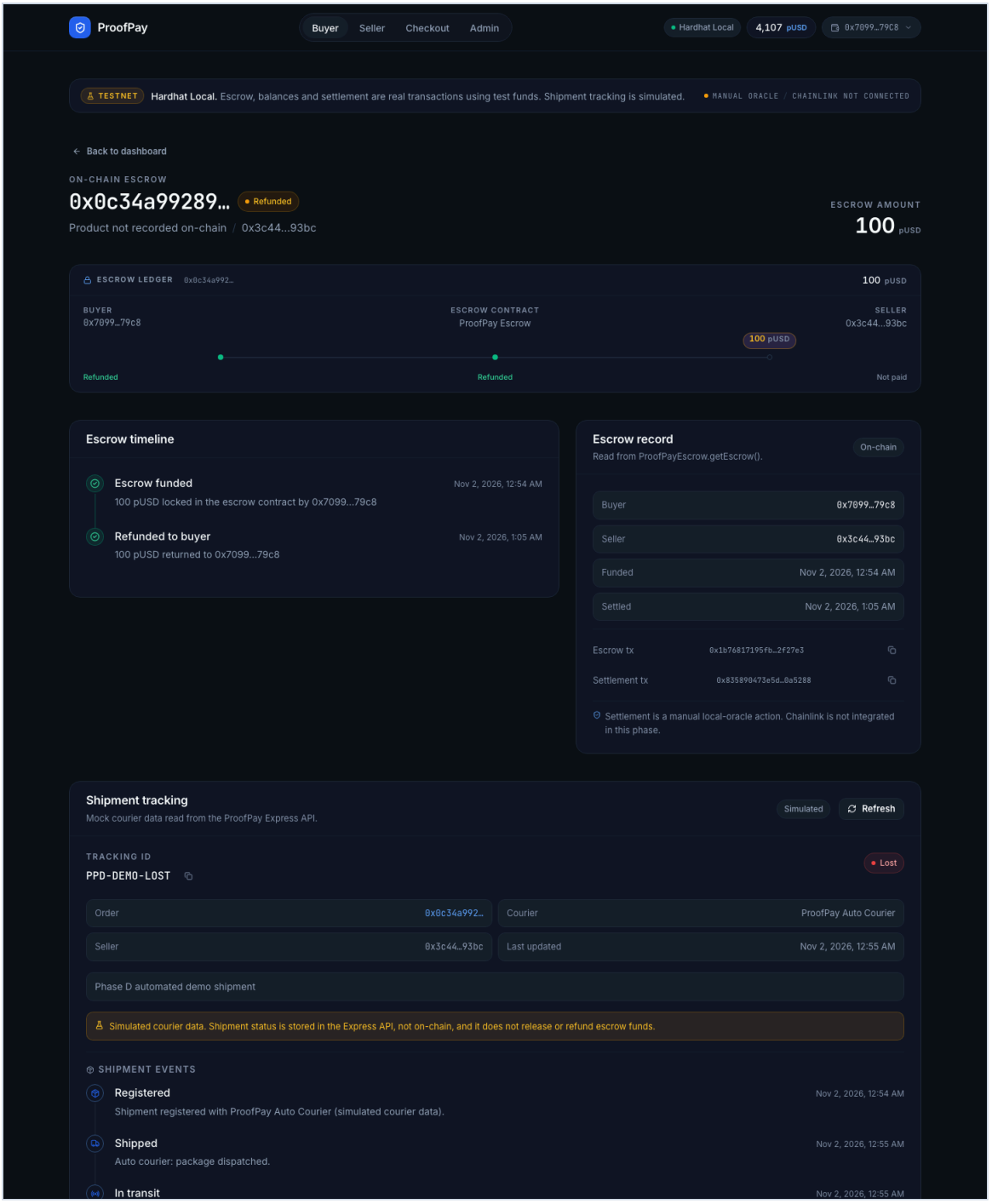


Figure 14: Order Tracking for the Lost Shipment — REFUNDED Escrow and Courier Event History

Figure 14 is the lost order after settlement. The escrow block on the left reports `Refunded` and the ledger shows the amount returning to the buyer; the record panel shows `settledAt` and the settlement transaction; the tracking panel shows the courier events that produced the decision, ending in "Auto courier: package reported lost". The page footer states that shipment status comes from the Express API and is not authoritative for settlement, which is accurate: the decision came from CRE, and the escrow acted only when the receiver instructed it.

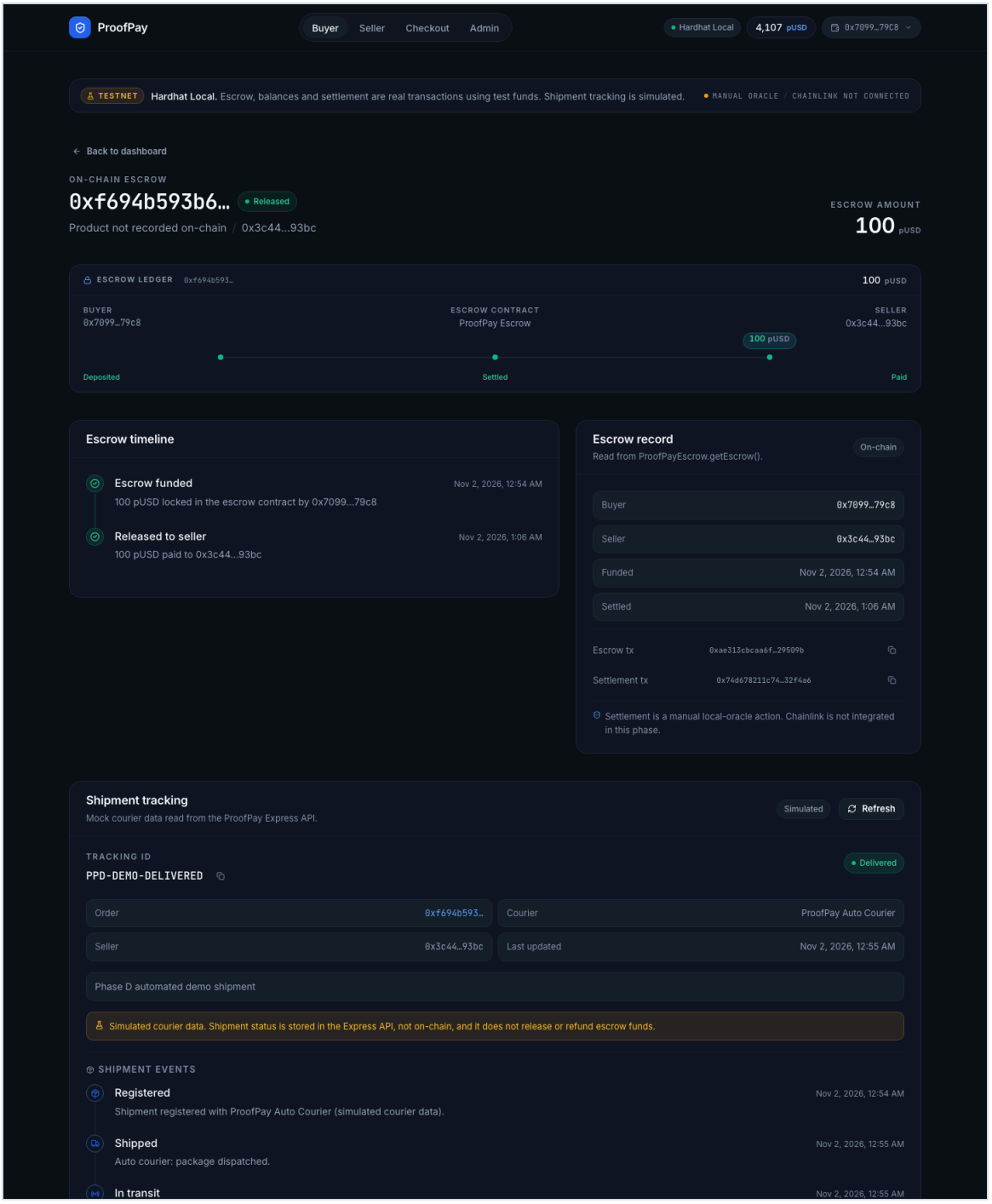


Figure 15: Order Tracking for the Delivered Shipment — RELEASED Escrow and Seller Payment

Figure 15 is the delivered order after release: status `Released`, the ledger showing 100 pUSD reaching the seller, the settlement transaction `0x74d6…f4a6` at block 30, and the courier timeline ending in "package delivered". The pair of figures demonstrates both terminal outcomes on the same interface.

12. SECURITY ANALYSIS AND LIMITATIONS

12.1 Access control

Three roles are separated deliberately. Ownership is OpenZeppelin's `Ownable`: the owner may set the escrow's oracle, the receiver's forwarder and manual oracle, and mint pUSD, but cannot move escrowed funds. The escrow's `oracle` is the receiver contract, and `onlyOracle` is the sole gate on both settlement functions; because the receiver is a contract, there is no key that can call the escrow directly. The receiver's `onlyAuthorized` accepts the forwarder, the manual oracle, or the owner when no forwarder is set — a deliberate allowance for local testing that should be removed before any deployment, since it means the owner can settle while `forwarder` is the zero address. In the local configuration the forwarder is zero and the manual oracle is the admin account, so in practice one key can settle through the receiver. That key cannot move tokens except through the escrow's own funded-order checks, which bounds the damage to premature settlement rather than theft of unrelated balances.

12.2 Reentrancy

Every state-changing function in the escrow is `nonReentrant`, using OpenZeppelin's `ReentrancyGuard`. The ordering is also correct independently of the guard: `createEscrow` writes the struct before pulling tokens, and both settlement functions set the terminal status and `settledAt` before transferring. A malicious token that attempted to re-enter would therefore find the order already terminal even if the guard were removed. The test suite includes a reentrant creation case that would fail if either defence were dropped.

12.3 Token safety

All token interaction goes through `SafeERC20`, so a token that returns `false` instead of reverting cannot silently fail. The escrow's constructor rejects a token address with no bytecode, which prevents a permanently useless escrow from being deployed by a typo. Because pUSD is the project's own contract it is well behaved, but the escrow is written so that a different, less cooperative ERC-20 would still be handled correctly.

12.4 Input validation

Validation is layered by design. The API rejects malformed order IDs, empty tracking IDs, unknown statuses and illegal transitions. The workflow rejects unknown shipment statuses, refuses to build a report for an order whose escrow is not funded, and verifies that the bytes it is about to publish decode to the intended order and outcome. The bridge re-checks the order ID pattern, the outcome vocabulary, the ABI decoding, the allowlist and the state file before preparing calldata. The receiver rejects a zero order ID, an unauthorised caller, a repeated report, an order already settled, an order that is not funded, and any outcome other than the two it supports. The escrow performs its own eleven checks. No single layer is trusted to be complete — which is the practical response to the distortion risk described in the literature review.

12.5 External data risks

The shipment API is the weakest link, and it is worth being precise about why. A wrong status produces a wrong but valid settlement: if a lost parcel is recorded as delivered, the seller is paid and the buyer's recourse is off-chain. A delayed status leaves an escrow funded past delivery, which is a liveness cost rather than a loss. A manipulated status is possible for anyone holding the mutation secret; the secret lives only in server-side configuration, mutating routes reject requests without it, and the API is not exposed publicly in this deployment, but the secret is a single point of compromise. API downtime fails safe — the workflow logs an HTTP error and settles nothing. The structural mitigation is that the API's status is only one input: the escrow's funding state is read independently from the chain, so a forged "delivered" claim cannot settle an order that was never funded or has already been paid.

12.6 Oracle and bridge security

The honest characterisation is that the trust model of this deployment is weaker than the trust model the architecture is designed for. In a DON deployment, report production and delivery are consensus functions and the receiver authenticates the forwarder; here, the workflow runs in a simulator on one machine and the report is delivered by a local bridge using an unlocked development account. A compromised bridge could suppress a report or replay one, though replay is blocked on chain, and it could submit a report it forged — the receiver would accept it because the caller is authorised. What the bridge cannot do is move funds outside the escrow's rules, settle an order that is not funded, or settle one twice. The configuration is also deliberately conservative: the bridge runs read-only by default, its allowlist scopes a run to named orders, its state file records every submission, it refuses to retry after an ambiguous failure, and the CRE workflow's own `writeReport` remains disabled. The absence of a private key in the repository is a further reduction of risk: the signer is the node's own unlocked account, available only on a local development chain.

12.7 Limitations

The following limitations are stated explicitly because they bound every claim in this report. The system runs only on a local Hardhat node with chain ID 31337; there is no public testnet or mainnet deployment, and no verified contract source on any block explorer. Nothing has been executed on a Chainlink DON: the CRE workflow is run through the official CLI simulator with `dryRun` enabled, and the local bridge is a relay, not decentralised consensus. The courier is a simulator — the tracking IDs, statuses and timestamps are generated by the project's own scheduler, and no integration with DHL, FedEx, UPS or any other carrier exists. pUSD is a prototype token with no reserve, no peg, no redemption and no regulatory status; it must not be described as a stablecoin. There has been no third-party security audit, no formal verification and no penetration test. The local infrastructure is development-grade: a single JSON file for persistence, a shared-secret header instead of a real authentication scheme, an unlocked account for settlement, and an owner-only fallback in the receiver that would need removal before deployment. Finally, the automated courier loop was verified as a status-advancement mechanism and its bridge invocation is implemented and configurable, but the end-to-end unattended run is not demonstrated in this report (TC-18 is marked not run).

13. CONCLUSION

This project built and demonstrated a working payment-protection system for high-value online commerce. Three Solidity contracts — a purpose-built ERC-20 token, a custody escrow with exactly two terminal transitions, and a receiver that authorises, decodes and de-duplicates settlement reports — were deployed on a local Hardhat chain and wired so that the receiver is the escrow's only settlement authority. A React application gives buyers, sellers and an administrator views derived from chain state rather than from a private database, an Express service holds the off-chain shipment record with validated transitions, and a Chainlink CRE workflow reads that record over HTTP, reads the escrow over EVM, decides the outcome, and encodes a report carrying the order, outcome, tracking hash and observation time. Each of the twelve objectives in Section 2 is met, and each is evidenced by a test, a log or a transaction.

The system was validated in layers rather than asserted. Thirty-one contract tests and seventeen backend tests pass, the front end type-checks and builds, and the workflow's decision table was exercised across all six statuses. More importantly, three settlements were executed on chain from CRE-generated reports and verified independently by reading the escrow, the receiver's mappings and token balances: a release at block 24, a refund at block 29 and a second release at block 30, each with receipt status 1, each moving exactly the escrowed 100 pUSD, and each advancing the chain by exactly one block. The lost-order run is the clearest single piece of evidence for the architecture, because CRE reported two shipments, the allowlist admitted one, and exactly one transaction resulted. The honest boundary is equally clear: the workflow ran in the official simulator with its on-chain write disabled, and a local bridge submitted the report, because the installed SDK exposes no chain selector for a local Hardhat node and no DON was available. The decision was always CRE's; the delivery was not.

The broader relevance is that payment protection is a data problem disguised as a custody problem. Smart contracts already solve custody convincingly, and ERC-20 tokens already give exact, auditable transfers; what decides whether a buyer is protected is the quality and traceability of the fact that triggers the transfer. Building ProofPay made that concrete in three ways. First, the hardest engineering work was not the contracts but the seam between an off-chain record and an on-chain consequence — extracting the right bytes, proving they decode to the intended order, and refusing to guess. Second, the useful security properties came from layering independent checks rather than from any single clever mechanism: the API validates transitions, the workflow re-reads the chain, the receiver enforces authorisation and replay, the escrow enforces funding. Third, intellectual honesty about the execution environment is itself a design feature: separating "real CRE execution" from "decentralised DON execution" forced a clearer architecture, in which the oracle is a replaceable component and the relay holds no decision authority. Those lessons transfer directly to any FinTech system that must let code act on facts that originate outside it.

14. FUTURE SCOPE

1. **Public testnet deployment.** Deploy the three contracts to a public test network, verify the sources on the explorer, and re-point the front end and the workflow configuration. This is the single change that would make the system independently inspectable, and it requires no contract modification because the addresses are configuration.
2. **Actual Chainlink DON workflow deployment.** Register the workflow with a DON, set the receiver's `forwarder` to the DON's forwarder address, and remove the owner fallback in `onlyAuthorized`. At that point the bridge becomes unnecessary and the report's authenticity rests on DON consensus rather than on a local account.
3. **Real courier integration.** Replace the simulated scheduler with adapters for carrier tracking APIs, normalising their statuses into the existing vocabulary. The workflow's HTTP capability already accommodates this; the work is in authentication, rate limits and mapping carrier-specific states such as "customs hold" onto the six statuses.
4. **Production-grade off-chain infrastructure.** Move shipment persistence from a JSON file to a transactional database, replace the shared secret with signed requests and per-client credentials, add an outbox so that status changes and settlement triggers cannot be lost, and expose the API over TLS with observability.
5. **Layer-2 deployment.** Move to an L2 to reduce the roughly 122,000 gas per settlement by one to two orders of magnitude, which is what makes small-ticket orders viable and allows more frequent oracle evaluation.
6. **Multiple payment tokens.** Support additional ERC-20 assets, or a token registry keyed by currency, so that buyers can pay in their preferred unit while the escrow keeps its single-asset accounting per order.
7. **Advanced dispute resolution.** Give `RETURNED` and contested deliveries a real path: an evidence submission window, an arbitration role with bounded authority, and a partial-refund outcome, all recorded on chain rather than resolved off it.
8. **Formal audit and verification.** Commission a third-party audit and apply formal methods to the escrow's state machine and the receiver's replay logic, since those are the two places where a defect would be financially exploitable.
9. **Monitoring and notification.** Alert on shipments that remain non-terminal past an expected window, on escrows funded but never shipped, and on workflow runs that fail or produce unverified payloads, so that operational silence is detectable.
10. **Scalability and analytics.** Batch oracle evaluation across orders, index events for dashboards, and add analytics on settlement latency and dispute rates — the metrics a payment-protection product is actually judged on.

15. REFERENCES

1. L. Albshaier, S. Almarri and M. Hafizur Rahman, "A Review of Blockchain's Role in E-Commerce Transactions: Open Challenges, and Future Research Directions," *Computers*, vol. 13, no. 1, art. 27, 2024, doi: 10.3390/computers13010027.
2. G. Caldarelli, "Overview of Blockchain Oracle Research," *Future Internet*, vol. 14, no. 6, art. 175, 2022, doi: 10.3390/fi14060175.
3. M. V. Nguyen, N. T. Le, D. Duong and Y. Li, "Blockchain Oracles: Implications for Smart Contracts in Legal Reasoning and Addressing the Oracle Problem," in *Proc. 12th Int. Symp. on Information and Communication Technology (SoICT '23)*, 2023, pp. 296–303, doi: 10.1145/3628797.3628870.
4. C. Allenbrand, "Smart contract-enabled consortium blockchains for the control of supply chain information distortion," *Blockchain: Research and Applications*, vol. 4, no. 3, art. 100134, 2023, doi: 10.1016/j.bcra.2023.100134.
5. Chainlink Labs, "Chainlink Compute Runtime Environment (CRE) — Overview," Chainlink Documentation. [Online]. Available: https://docs.chain.link/cre
6. Chainlink Labs, "CRE SDK and CLI Reference," Chainlink Documentation. [Online]. Available: https://docs.chain.link/cre/reference
7. Ethereum Foundation, "Solidity Documentation, version 0.8.28." [Online]. Available: https://docs.soliditylang.org
8. Nomic Foundation, "Hardhat Documentation." [Online]. Available: https://hardhat.org/docs
9. OpenZeppelin, "OpenZeppelin Contracts 5.6.1 API Reference." [Online]. Available: https://docs.openzeppelin.com/contracts/5.x/
10. F. Vogelsteller and V. Buterin, "EIP-20: Token Standard," Ethereum Improvement Proposals, 2015. [Online]. Available: https://eips.ethereum.org/EIPS/eip-20

APPENDIX A — DEPLOYMENT AND TRANSACTION LEDGER

Every transaction below was read from the local Hardhat node over RPC and is reproducible with `eth_getTransactionReceipt` against `http://127.0.0.1:8545`. Gas figures are actual consumption. No transaction from outside this ledger is claimed anywhere in this report.

Table A.1: Contract Deployments and Authorisation

Block	Action	Transaction hash	Gas	Result
1	Deploy ProofPayUSD	`0xb10fd1938748aa3d1cd691525cccd3a6857772b93dbc3e9fa44e3179cd1c0116`	618,390	`0x5FbD…0aa3`
2	Deploy ProofPayEscrow	`0x126d912777073a40828533d031b5c25274b0b33650a8b6d5f8cd38295b573fb1`	815,030	`0xe7f1…0512`
3	setOracle (escrow)	`0xe78ec9c5daaeb8a44ecbfb7cee7c4e3a9837585dff0f1733a611639d462b0ba9`	47,639	oracle set
18	Deploy ProofPayCREReceiver	`0x2c1b12ddabf5a951c26eaef23f2bbaa8c3db0df6210affa9506bf46b0991ee7d`	880,716	`0xB7f8…4F5e`
19	setOracle (escrow → receiver)	Recorded in the deployment log; hash not separately captured	30,539	oracle = receiver

Table A.2: Demonstration Settlements

Block	Order	Transaction hash	Gas	Report ID
24	`0x4d07…aaf3`	`0x78af4106dfa323c13640f75a0e0fd8ec54320742cc5c6ba577adc45fc712bd6c`	122,471	`0x12ccb195…c751c`
29	`0x0c34…1152`	`0x835890473e5d61dc4a1c8a27fdefd94f0777df26169554f1420534f0670a5288`	122,425	`0x774fb352…25ea4`
30	`0xf694…c26434`	`0x74d678211c74efda145b9c712adae35bcd5a35616275ec8d76d0dcc3a232f4a6`	122,471	`0x320fd2a9…3c3bf`

Ledger state at block 32 (read-only snapshot). Total pUSD supply 1,010,100; escrow contract holding 7,289 pUSD across the three still-funded orders; buyer balance 608 pUSD; seller balance 5,993 pUSD; owner balance 996,210 pUSD; nine escrows in total, of which four are Released, two are Refunded and three remain Funded. The buyer figure reflects a later, unrelated 3,499 pUSD order created through the interface after the three settlements were verified; the post-settlement balances recorded in Figure 6 are buyer 4,107 pUSD and seller 5,993 pUSD, which is the state in which the demonstration ended.

APPENDIX B — LIST OF FIGURES AND TABLES

Table B.1: List of Figures

Figure	Title
1	Overall Architecture of the Proposed ProofPay System
2	End-to-End Shipment Verification and Settlement Workflow
3	Contract Interaction Sequence for a Single Escrow Order
4	Shipment Lifecycle and the Escrow State It Drives
5	Contract Test Suite Output (31 Passing) and Backend Test Summary (17 Passing)
6	On-Chain Verification of the Three Settlements Read Back from Hardhat
7	CRE Workflow Simulation Output Showing the Structured Report Records and Dry-Run Skip
8	Local Execution Bridge in Dry-Run Mode — Preflight Passed, Calldata Prepared, Nothing Broadcast
9	ProofPay Landing Page
10	Buyer Dashboard with Live pUSD Balance and Escrow Activity
11	Checkout — Approve and Fund an Escrow
12	Seller Dashboard with Incoming Escrows and Settlement Status
13	Admin Dashboard — Oracle Wiring, Shipment Simulator and Receiver Panel
14	Order Tracking for the Lost Shipment — REFUNDED Escrow and Courier Event History
15	Order Tracking for the Delivered Shipment — RELEASED Escrow and Seller Payment

Table B.2: List of Tables

Table	Title
1	Linkage of the Problem Statement
2	Details of the External Source
3	Literature Review Summary
4	Existing Systems
5	Smart Contract Design
6	Deployment Details
7	Test Cases
8	Gas Analysis
9	Executed CRE-Mediated Settlements
A.1	Contract Deployments and Authorisation
A.2	Demonstration Settlements
B.1	List of Figures
B.2	List of Tables

End of report. All figures are reproduced from the ProofPay repository or from captures of its running instance; all transaction hashes and gas figures were read from the local Hardhat node at the block heights stated.